Federated Learning for Multi-Agent AI: A Comparative Study Across Vehicular Communication, Network Security and UAV Trajectory Planning

SEMINAR REPORT

Submitted by

Akhilesh K S

SNM23CA006

to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Technology

in

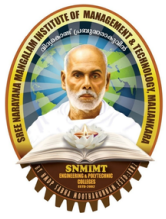
Computer Science and Engineering (Artificial Intelligence)

Under the guidance of

Ms. Chinju V C

Assistant Professor

Dept. of CSE



Department of Computer Science and Engineering (Artificial Intelligence)

SREE NARAYANA MANGALAM INSTITUTE OF MANAGEMENT AND TECHNOLOGY

MALIANKARA

October 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE)

SREE NARAYANA MANGALAM INSTITUTE OF MANAGEMENT & TECHNOLOGY, MALIANKARA



CERTIFICATE

This is to certify that the seminar report entitled **Federated Learning for Multi-Agent AI: A Comparative Study Across Vehicular Communication, Network Security and UAV Trajectory Planning** *submitted by* **Akhilesh K S** *(***SNM23CA006***), to the* **Department of Computer Science and Engineering (Artificial Intelligence), SNM Institute of Management and Technology, Maliankara**, *in partial fulfilment of the requirement for the award of B.Tech Degree in Computer Science and Engineering (Artificial Intelligence) is a bonafide record of the work carried out by the student.*

Guide	Head of the Department
Ms. Chinju V C	**Ms. Lakshmi Anand**
Asst. Professor	**Asst. Professor**
Dept. of CSE	**Dept. of CSE**

Coordinator

Ms. Surya T S
Asst. Professor
Dept. of CSE

Declaration

I undersigned hereby declare that the seminar report **Federated Learning for Multi-Agent AI: A Comparative Study Across Vehicular Communication, Network Security and UAV Trajectory Planning**, submitted for partial fulfilment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by me under the supervision of **Ms. Chinju V C**. This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Maliankara **Akhilesh K S**

Date: 30/09/2026 **SNM23CA006**

Acknowledgement

Firstly, I thank God, the Almighty, with whose blessings, this seminar has been successfully completed.

I wish to give deep sense of acknowledgement to our Principal **Dr. Atmaram P**, for his constant encouragement and valuable advice throughout the course.

I am extremely grateful to **Ms. Lakshmi Anand**, Head of the Department of Computer Science and Engineering, for the valuable guidance through this humble endeavor. The highly enterprising attitude, patience for listening to my doubts, timely suggestions and guidance made my seminar a so-called reality in stipulated time. I would also like to thank our seminar coordinator **Ms. Surya T S**, Assistant Professor, Department of Computer Science and Engineering and my seminar guide **Ms. Chinju V C**, Assistant Professor, Department of Computer Science and Engineering for their gracious of encouragement and the valuable assistance.

I express my sincere gratitude to all the faculty members of the department of Computer Science and Engineering (Artificial Intelligence) for their co-operation and support to the seminar.

I also express my sincere gratitude to all my friends and my parents for their co-operation and constant inspiration.

Akhilesh K S

Abstract

Multi-agent artificial intelligence systems increasingly need to learn collaboratively from decentralised experience without exposing raw, often privacy-sensitive, local data to a central authority or to one another. Federated learning offers exactly this capability, and its fusion with multi-agent reinforcement learning—known as Federated Multi-Agent Deep Reinforcement Learning (FedMARL)—has become an active research direction across wireless networks, cybersecurity, and aerial robotics. This seminar presents a comparative study of four recent works that apply federated multi-agent learning to distinct real-world coordination problems.

Shabir et al., the base paper, formulate task offloading in vehicular fog computing as a multi-agent problem in which every vehicle trains a local asynchronous advantage actor-critic model and a federation of fog servers aggregates these into a global model, cutting task residence time by 58.6% over greedy and stochastic offloading. Andong and Min propose FERMI-6G, where 6G edge agents use deep recurrent Q-networks to jointly choose task offloading, channel access, and CPU frequency, and synchronise through secure aggregation based on elliptic-curve Diffie–Hellman masking. Muller et al. survey federated multi-agent deep learning for distributed wireless sensing, covering intrusion detection, poisoning and backdoor threats, and federated control of UAV trajectories. Mohammadabadi et al. present ComDML, a serverless scheme in which slower agents offload part of their training workload to faster peers, reducing training time by up to 71%.

Comparing these systems across agent design, coordination structure, robustness to heterogeneity, and real-world deployability reveals a consistent pattern: federating locally trained models rather than raw data lets systems as different as connected vehicles, edge devices, sensor networks, and UAV swarms reach faster, more robust joint behaviour than independent learning—at the cost of coordination overhead and, where a server is used, a single point of trust. The study concludes that hybrid FedMARL architectures, combining domain-specific agent design with trust-aware aggregation, are the most promising direction for scalable, privacy-preserving multi-agent AI.

Keywords: Multi-Agent Artificial Intelligence; Federated Learning; Federated Multi-Agent Deep Reinforcement Learning; Vehicular Fog Computing; Secure Aggregation; 6G Edge Networks; Network Security; UAV Trajectory Planning; Decentralised Learning.

Contents

Acknowledgement **I**

Abstract **II**

List of Figures **VII**

List of Tables **VIII**

List of Abbreviations **IX**

1 Introduction **1**
1.1 General Background 2
1.2 Motivation 3
1.3 Problem Statement 3
1.4 Objectives 3
1.5 Scope 4
1.6 Advantages 4
1.7 Organization of the Report 4

2 Theoretical Foundations **5**
2.1 Reinforcement Learning 5
2.2 Deep Reinforcement Learning 6
2.3 Multi-Agent Reinforcement Learning 7
2.4 Federated Learning 8
2.5 Decentralised and Split Learning 9
2.6 Federated Multi-Agent Reinforcement Learning 9
2.7 Privacy and Security in Federated Learning 10
2.8 Edge Computing and Wireless Context 11
2.9 Evaluation Metrics 12
2.10 Taxonomy of Approaches 12
2.11 Chapter Summary 13

3 Literature Survey **14**
3.1 A Federated Multi-Agent Deep Reinforcement Learning for Vehicular Fog Computing 14
3.1.1 Problem Statement and Motivation 14
3.1.2 System Model and Problem Formulation 15
3.1.3 Multi-Agent A3C and Two-Tier Federation 16

3.1.4 Experimental Setup . . . 17
3.1.5 Results and Analysis . . . 17
3.1.6 Discussion . . . 19
3.2 Federated Multi-Agent Reinforcement Learning for Privacy-Preserving and Energy-Aware Resource Management in 6G Edge Networks . . . 19
3.2.1 Problem Statement and Motivation . . . 19
3.2.2 System Model and Reward . . . 20
3.2.3 Secure Aggregation and the FERMI-6G Algorithm . . . 21
3.2.4 Experimental Setup . . . 21
3.2.5 Results and Analysis . . . 22
3.2.6 Discussion . . . 23
3.3 Federated Multi-Agent Deep Learning and Neural Networks for Advanced Distributed Sensing in Wireless Networks . . . 24
3.3.1 Scope, Background, and Reference Architecture . . . 24
3.3.2 Advanced Techniques . . . 25
3.3.3 Applications: UAV Networks, Sensing, and Security . . . 26
3.3.4 Challenges and Future Directions . . . 26
3.3.5 Discussion . . . 27
3.4 Communication-Efficient Training Workload Balancing for Decentralized Multi-Agent Learning . . . 27
3.4.1 Problem Statement and Motivation . . . 27
3.4.2 Local-Loss Split Training and Optimisation . . . 28
3.4.3 Decentralised Pairing and Training Workflow . . . 29
3.4.4 Experimental Setup . . . 30
3.4.5 Results and Analysis . . . 30
3.4.6 Discussion . . . 32
3.5 Summary of the Reviewed Papers . . . 32

4 Discussion & Comparison **33**
4.1 Analytical Focus and Methodology . . . 33
4.2 Agent Design and Coordination Structure . . . 33
4.3 Robustness to Heterogeneity . . . 34
4.4 Security and Privacy . . . 35
4.5 Scalability, Communication, and Results . . . 36
4.6 Real-World Deployability . . . 36
4.7 Overall Comparative Insights . . . 36

5 Applications **37**
5.1 Connected and Autonomous Vehicles . . . 37
5.2 6G Edge Networks and Mobile Edge Computing . . . 37
5.3 Network Security and Intrusion Detection . . . 37

5.4 UAV Trajectory Planning and Aerial Networks 38
5.5 Sensing, Smart Cities, and Collaborative Training 38

6 Challenges and Future Directions **39**
6.1 Challenges Identified in the Reviewed Works 39
6.2 Cross-Cutting Challenges 39
6.3 Future Research Directions 40
6.4 Ethical and Societal Considerations 40
6.5 Summary 40

7 Conclusion **41**

References **43**

List of Figures

1.1.1 Three paradigms for learning across many agents: centralised, federated, and decentralised (peer-to-peer) 2

2.3.1 Multi-agent reinforcement learning under partial observability: every agent acts on local observations, but the environment responds to the joint action 7

2.6.1 Federated multi-agent reinforcement learning: agents learn locally from their own interaction and periodically fuse model parameters through an aggregator 10

2.7.1 Secure aggregation with pairwise masks: each individual upload is hidden, but the masks cancel in the sum 11

2.10.1 A taxonomy of federated multi-agent learning and the positions of the reviewed works 13

3.1.1 Two-tier federated learning framework: vehicles in the lower tier train local actor-critic models, and the fog-server federation in the upper tier maintains the global network [1] 16

3.1.2 Residence time and end-to-end delay versus task arrival rate for GA, SA, and the proposed DRL scheme [1] 18

3.1.3 Delivery ratio versus arrival rate, and one-hop deliveries, two-hop deliveries, and failures at $\lambda = 200$ [1] 18

3.1.4 Workload distribution and utilisation ratio at $\lambda = 200$ tasks/s [1] 19

3.2.1 System architecture of FERMI-6G: each agent observes local state, selects application- and MAC-layer actions with a DRQN, stores prioritised experience, updates locally, and joins the secure federated protocol [2] . . 20

3.2.2 Reliability per episode: (a) FERMI-6G, (b) centralised RL with cross-layer optimisation, (c) Fed-MARL without cross-layer optimisation, (d) centralised RL [2] 23

3.2.3 Scalability, energy efficiency, and throughput of the four frameworks, plotted from Table 3.1 (Central-CL: centralised RL with cross-layer optimisation) 24

3.3.1 Conceptual architecture for distributed sensing, wireless communication, and multi-agent deep learning (redrawn from [3]) 25

3.4.1 Training with and without workload balancing: offloading part of Agent 1's model to Agent 2 fills Agent 2's idle time and shortens the round [4] 28
3.4.2 ComDML training round: agents 1 and 2 offload work to faster agents 5 and 4, agent 3 trains independently, and all models are aggregated with AllReduce [4] 30
3.4.3 Training time to target accuracy with 10 heterogeneous agents, plotted from Table 3.4 31

4.2.1 Qualitative design space of the reviewed systems: degree of decentralisation versus protection of individual model updates 35

List of Tables

2.1 6G service classes and the task profiles used to simulate them in [2] 12
2.2 Evaluation metrics used in the reviewed works 12

3.1 Framework comparison, mean ± standard deviation [2] 22
3.2 Core learning paradigms and their suitability for wireless systems (adapted from [3]) 26
3.3 Two-agent training time (s) on CIFAR-10 with ResNet-56 for different numbers of offloaded layers [4] 31
3.4 Training time (s) with 10 agents to reach the target accuracy [4] 31
3.5 Summary of the reviewed works 32

4.1 Methodological comparison of the reviewed works 34
4.2 Threats and protection mechanisms across the reviewed works 35

List of Abbreviations

6G	Sixth-Generation Mobile Networks
A3C	Asynchronous Advantage Actor-Critic
AES	Advanced Encryption Standard
AI	Artificial Intelligence
AirComp	Over-the-Air Computation
CPU	Central Processing Unit
CTDE	Centralised Training with Decentralised Execution
Dec-POMDP	Decentralised Partially Observable Markov Decision Process
DL	Deep Learning
DML	Decentralised Multi-Agent Learning
DNN	Deep Neural Network
DP	Differential Privacy
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
DRQN	Deep Recurrent Q-Network
ECDH	Elliptic-Curve Diffie–Hellman
eMBB	Enhanced Mobile Broadband
FedAvg	Federated Averaging
FedMARL	Federated Multi-Agent Reinforcement Learning
FL	Federated Learning
FRL	Federated Reinforcement Learning
GA	Greedy Algorithm
GNN	Graph Neural Network
IDS	Intrusion Detection System
IID	Independent and Identically Distributed
IoT	Internet of Things
ISAC	Integrated Sensing and Communication
LSTM	Long Short-Term Memory
MAC	Medium Access Control
MADL	Multi-Agent Deep Learning

MARL	Multi-Agent Reinforcement Learning
MDP	Markov Decision Process
MEC	Mobile Edge Computing
mMTC	Massive Machine-Type Communications
NOMA	Non-Orthogonal Multiple Access
O-RAN	Open Radio Access Network
POMMDP	Partially Observable Multi-Agent Markov Decision Process
QoS	Quality of Service
RL	Reinforcement Learning
SA	Stochastic Selection Algorithm
SGD	Stochastic Gradient Descent
SL	Split Learning
TD	Temporal Difference
UAV	Unmanned Aerial Vehicle
URLLC	Ultra-Reliable Low-Latency Communications
V2P	Vehicle-to-Parked-Vehicle
V2V	Vehicle-to-Vehicle
V2X	Vehicle-to-Everything
WSN	Wireless Sensor Network

Chapter 1

Introduction

Artificial intelligence is moving from single, isolated models towards **multi-agent systems** in which many autonomous entities—vehicles, base stations, edge servers, sensors, and unmanned aerial vehicles (UAVs)—observe their surroundings, make decisions, and learn at the same time. A connected car decides whether to execute a computation on board or offload it to a roadside fog server; a 6G edge device decides which channel to use and how fast to run its processor; a UAV in a swarm decides where to fly next. These decisions interact through shared spectrum, compute, and space, and no agent sees the whole system [3].

Reinforcement Learning (RL) learns such sequential decisions from interaction and reward, and its deep and multi-agent extensions (DRL and MARL) can approximate complex policies over very large state spaces [1]. A multi-agent learner, however, sees only a small, biased part of the environment, while the obvious remedy—collecting every agent's experience on one server—is often impossible because raw data are private, wireless links are limited, and a central server is a single point of failure and attack [2]. **Federated Learning** (FL) resolves this tension: each participant trains on its own data and shares only model parameters, which an aggregator combines into a global model. When the local learners are RL agents, the result is **Federated Multi-Agent Deep Reinforcement Learning** (FedMARL), in which agents explore different parts of a shared environment and periodically fuse their policies [1, 3].

This seminar presents a comparative study of four recent works spanning vehicular communication, 6G edge networks, network security and UAV networks, and decentralised training. Paper [1], the base paper, by Shabir et al., proposes a federated multi-agent DRL framework for **vehicular fog computing**, in which vehicles train asynchronous

advantage actor-critic (A3C) offloading policies and a federation of fog servers aggregates them. Paper [2], by Andong and Min, introduces **FERMI-6G**, in which edge agents use deep recurrent Q-networks for cross-layer decisions and synchronise through **secure aggregation** based on elliptic-curve Diffie–Hellman key exchange. Paper [3], by Muller et al., **surveys** federated multi-agent deep learning for distributed wireless sensing, including intrusion detection, attacks on federated learning, and UAV networks. Paper [4], by Mohammadabadi et al., presents **ComDML**, a serverless scheme that balances training workload between fast and slow agents.

1.1 General Background

Learning across many devices has passed through three stages, illustrated in Figure 1.1.1. In the **centralised** stage, devices upload raw data to a cloud server that trains one model; this gives a complete view but consumes bandwidth, adds latency, and concentrates sensitive data [1]. In the **federated** stage, devices train locally and a server aggregates model updates, often through hierarchical tiers such as vehicles, fog servers, and cloud [1, 3]; data movement falls sharply, but a trusted aggregator is still required and heterogeneous devices slow training down. In the **decentralised** stage, agents exchange models directly through gossip, AllReduce, or consensus protocols, removing the single point of failure but making coordination harder [4].

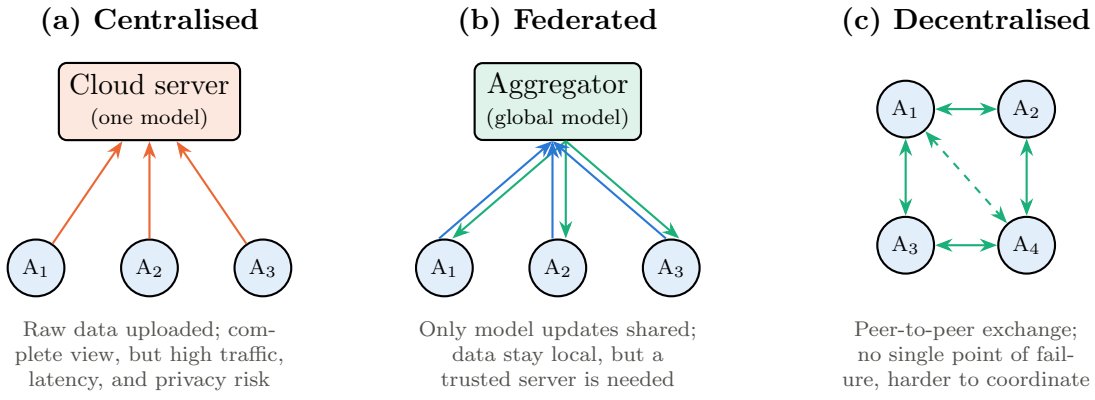


Figure 1.1.1: Three paradigms for learning across many agents: centralised, federated, and decentralised (peer-to-peer)

At the same time, the *kind* of model being federated has changed. Early work federated supervised predictors; recent work increasingly federates **decision-making policies**, so that each agent is a reinforcement learner and what is shared is a policy or value network [1, 2]. Wireless systems are a key application: 6G networks are expected to be ultra-dense, to integrate sensing with communication, to push computation to the edge, and to include aerial nodes, while serving ultra-reliable low-latency (URLLC), enhanced broadband (eMBB), and massive machine-type (mMTC) traffic with very different requirements [2, 3].

1.2 Motivation

Three needs motivate this study. First, **decentralised decision-making**: vehicular networks change topology every second, edge devices share congested spectrum on limited batteries, and UAV swarms have intermittent connectivity, so a central controller with full observability is unavailable or too slow [1, 2]. Second, **privacy and security**: offloading requests reveal location, resource usage reveals behaviour, and traffic used for intrusion detection is confidential; FL keeps these data local, but updates can still leak information and malicious participants can poison the shared model [2, 3]. Third, **efficiency**: agents differ in compute, links, and energy, and training that waits for the slowest agent wastes the capacity of the others [4].

1.3 Problem Statement

Given agents that observe only part of a shared, dynamic environment, hold private data, and differ in their resources, learn policies or models that achieve good collective performance—low latency and energy, high reliability, and fairness—while keeping raw data local, protecting individual updates, and limiting communication and training time.

1.4 Objectives

The objectives of this seminar are:

- To introduce reinforcement learning, deep and multi-agent RL, and federated learning, on which FedMARL is built.
- To review in depth four recent works on federated multi-agent learning for vehicular

fog computing, 6G edge resource management, distributed wireless sensing, and decentralised training.

- To examine the security and privacy mechanisms used or discussed, including secure aggregation, differential privacy, and defences against poisoning.
- To compare the approaches and identify open challenges and future research directions.

1.5 Scope

The seminar covers task offloading in vehicular fog computing [1], cross-layer resource management in 6G edge networks [2], federated multi-agent learning for sensing, security, and UAV networks as surveyed in [3], and workload balancing for serverless training [4], together with the algorithms, simulators, datasets, and metrics these works use.

1.6 Advantages

Federated multi-agent learning offers several advantages over centralised and independent learning:

- **Privacy by Design:** raw data such as positions, traffic, and sensor readings never leave the device.
- **Reduced Communication:** periodic model updates need far less bandwidth than streaming raw data.
- **Faster Convergence:** agents exploring different parts of the environment pool their knowledge.
- **Cross-Domain Applicability:** the same principles apply to vehicles, edge devices, sensors, and UAVs.

1.7 Organization of the Report

Chapter 2 introduces the theoretical foundations. **Chapter 3** reviews the four works in detail. **Chapter 4** compares the approaches, **Chapter 5** discusses applications, **Chapter 6** presents challenges and future directions, and **Chapter 7** concludes the report.

Chapter 2

Theoretical Foundations

This chapter introduces the concepts on which the reviewed works build: reinforcement learning and its deep and multi-agent extensions, federated and decentralised learning, their combination in FedMARL, the privacy and security mechanisms used to protect federated training, the edge-computing context, and the evaluation metrics.

2.1 Reinforcement Learning

Reinforcement learning formalises sequential decision-making as a **Markov Decision Process** (MDP), the tuple $\langle \mathcal{S}, \mathcal{A}, P, r, \gamma \rangle$ of states, actions, transition probabilities $P(s' \mid s, a)$, rewards $r(s, a)$, and discount factor $\gamma \in (0, 1]$ [1]. At each step the agent observes s_t, chooses a_t from a policy $\pi(a \mid s)$, receives r_t, and moves to s_{t+1}. It seeks to maximise the expected discounted return

$$R_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k}. \tag{2.1}$$

The state value $V^\pi(s) = \mathbb{E}_\pi[R_t \mid s_t = s]$ and action value $Q^\pi(s, a) = \mathbb{E}_\pi[R_t \mid s_t = s, a_t = a]$ measure how good a policy is. **Q-learning** estimates the optimal action value with the temporal-difference (TD) update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \Big[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \Big], \tag{2.2}$$

where the bracketed term is the **TD error**. Exploration usually follows an ϵ-greedy rule whose random-action probability decays during training, for example from 1.0 to 0.05 with factor 0.995 in [2].

2.2 Deep Reinforcement Learning

Tables become infeasible when the state space is large, as in vehicular and wireless environments with many queues, positions, and channels [1]. The **Deep Q-Network** (DQN) [8] approximates $Q(s, a; \theta)$ with a neural network and minimises the squared TD error

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-) - Q(s_t, a_t; \theta)\big)^2\Big], \tag{2.3}$$

where θ^- is a periodically updated **target network**. Transitions are stored in an **experience replay** buffer and sampled in mini-batches; **prioritised replay** samples transitions with large TD error more often [2]. Under partial observability a single observation is not enough, so the **Deep Recurrent Q-Network** (DRQN) [9] inserts a Long Short-Term Memory (LSTM) layer that summarises the observation history in a hidden state, $h_t = \mathrm{LSTM}(h_{t-1}, o_t)$, and estimates $Q(h_t, a; \theta)$. Paper [2] gives every agent a DRQN with sequence length 20.

Actor-critic methods learn the policy $\pi(a \mid s; \theta)$ directly (the *actor*) and use a value network $V(s; w)$ (the *critic*) as a baseline. The actor follows the policy gradient $\nabla_\theta \log \pi(a_t \mid s_t; \theta)\, A(s_t, a_t)$, where the **advantage** with a one-step bootstrap is

$$A(s_t, a_t; \theta, w) = r_t + \gamma V(s_{t+1}; w) - V(s_t; w), \tag{2.4}$$

and an **entropy** bonus $\beta \nabla_\theta H(\pi(s_t; \theta))$ keeps the policy exploratory; the critic minimises A^2 [1]. The **Asynchronous Advantage Actor-Critic** (A3C) algorithm [6] runs several learners in parallel environments, each of which applies its gradients asynchronously to a shared global network and copies the global parameters back. Parallel exploration decorrelates updates, removing the need for replay. This structure—local learners, a shared global model, and asynchronous parameter exchange—is very close to federated learning, which is why paper [1] builds on it.

2.3 Multi-Agent Reinforcement Learning

With several learning agents, the MDP generalises to a **Markov game** in which transitions and each agent's reward depend on the *joint* action. Agents may be cooperative, competitive, or mixed, and homogeneous or heterogeneous. If each agent receives only a local observation o_i, the model is a partially observable Markov game, or a **Decentralised POMDP** (Dec-POMDP) when agents share a team reward [3]. Paper [2] formulates 6G resource management as a partially observable multi-agent MDP (POMMDP). In wireless and edge networks, agents map naturally onto network entities—user devices, vehicles, base stations, access points, edge servers, network slices, and UAVs. Their observations include channel state, queue lengths, battery level, age of information, and sensing features, and their actions include transmit power, channel selection, beam choice, trajectory, and offloading decisions. Because these entities share spectrum and compute, one agent's action changes the environment of the others through interference, congestion, and load [3]. Figure 2.3.1 illustrates the interaction loop.

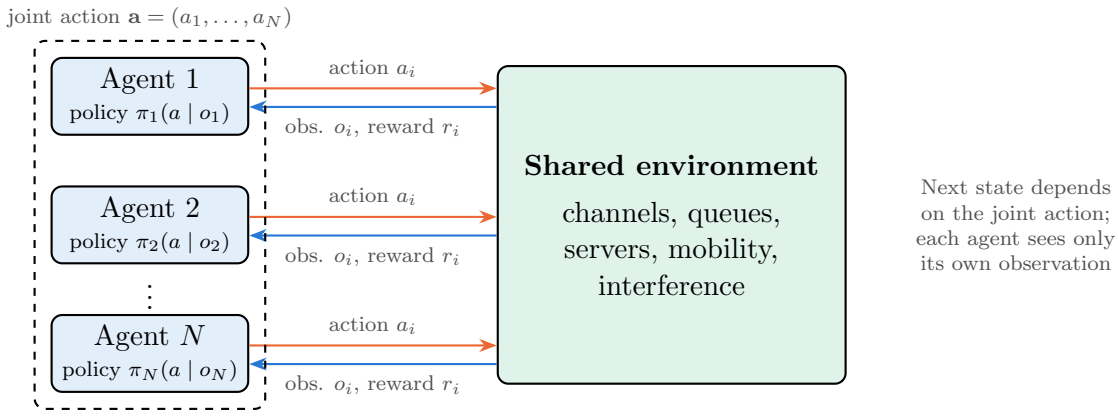


Figure 2.3.1: Multi-agent reinforcement learning under partial observability: every agent acts on local observations, but the environment responds to the joint action

MARL faces **non-stationarity** (each agent's environment changes as the others learn), **scalability** (the joint action space grows exponentially, which **mean-field** methods address by replacing many neighbours with their average influence), and **credit assignment** under shared rewards [3]. A common compromise is **Centralised Training with Decentralised Execution** (CTDE) [10]: training may use global information from a simulator or network telemetry, while each agent acts only on its own observations. FedMARL sits

between CTDE and fully independent learning: agents learn independently, but periodic model fusion gives them a shared, more global view without centralising data.

2.4 Federated Learning

Federated learning trains a shared model across K clients whose data remain local. Client i holds N_i samples with local objective $f_i(w) = \frac{1}{N_i}\sum_{n=1}^{N_i} \ell((x_n, y_n); w)$, and the global objective is the data-weighted average [4]

$$\min_w f(w) = \sum_{i=1}^{K} \frac{N_i}{N} f_i(w), \qquad N = \sum_{i=1}^{K} N_i. \tag{2.5}$$

The standard algorithm, **Federated Averaging** (FedAvg) [5], shown in Algorithm 1, alternates local stochastic gradient descent on each client with weighted averaging on the server.

Algorithm 1 Federated Averaging (FedAvg)

Require: clients $i = 1, \ldots, K$ with data sizes N_i; rounds R; local epochs E; learning rate η

Initialise global model w^0
for round $r = 0$ to $R - 1$ **do**
  Send w^r to a subset $\mathcal{C}_r$ of available clients
  for each client $i \in \mathcal{C}_r$ **in parallel do**
    $w_i \leftarrow w^r$; run E epochs of $w_i \leftarrow w_i - \eta \nabla f_i(w_i; b)$ over mini-batches b
  end for
  $w^{r+1} \leftarrow \sum_{i \in \mathcal{C}_r} \frac{N_i}{\sum_{j \in \mathcal{C}_r} N_j} w_i$
end for
return w^R

Real deployments add three complications. **Statistical heterogeneity**: client data are rarely IID, and non-IID data (often simulated with a Dirichlet distribution, concentration 0.5 in [4]) make local models drift apart. **System heterogeneity**: in synchronous FL every round lasts as long as the slowest client, the **straggler problem**; common remedies select fewer clients, drop the slowest, or vary local work as in FedProx [4]. **Communication constraints**: model uploads compete with user traffic for spectrum and power, so client selection, bandwidth and power allocation, and quantisation must be co-designed

with learning [3]. **Hierarchical FL** aggregates first over short links (for example vehicle to fog) and only occasionally at the cloud, and **over-the-air computation** lets the wireless channel itself sum simultaneous transmissions, at the cost of fading-induced errors [1, 3]. Finally, aggregation may be **synchronous**, with every round waiting for all selected clients, or **asynchronous**, with the global model updated whenever any client reports. Asynchronous aggregation, used by A3C-style training in [1], tolerates slow and intermittently connected clients but must cope with *stale* updates computed from an older global model; synchronous aggregation, used in [2], is simpler to analyse and secure but is paced by the slowest participant.

2.5 Decentralised and Split Learning

Decentralised learning removes the aggregator. In **gossip learning** agents average with random neighbours; in **BrainTorrent** they take turns as aggregator; and with **AllReduce** all agents cooperatively compute the average so that each ends with the same model [4]. For K agents and a model of b bytes, ring and recursive halving-and-doubling AllReduce both send and receive $2\frac{K-1}{K}b$ bytes per agent, but need $2(K-1)$ and $2\log_2 K$ communication steps respectively. **Split learning** (SL) divides a network at a cut layer so that a weak device trains only the first layers and sends intermediate activations to a stronger party; because classical SL waits for returned gradients in every step, **local-loss-based split training** attaches a small auxiliary network to the device side so both sides can update in parallel [4].

2.6 Federated Multi-Agent Reinforcement Learning

Federated reinforcement learning applies federated aggregation to RL agents: each agent trains a policy or value network on trajectories it collects locally, and only network parameters are shared [3]. When the agents also interact in a shared environment, the setting is **FedMARL** (Figure 2.6.1). Its key design choices are *what* is shared (actor-critic weights in [1], DRQN weights in [2]), *when* (every episode in [1], every K episodes in [2]), whether aggregation is synchronous or asynchronous, *who* aggregates (a fog server in [1], a server seeing only masked updates in [2], all peers via AllReduce in [4]), and *which* agents participate (only sufficiently charged ones in [2]).

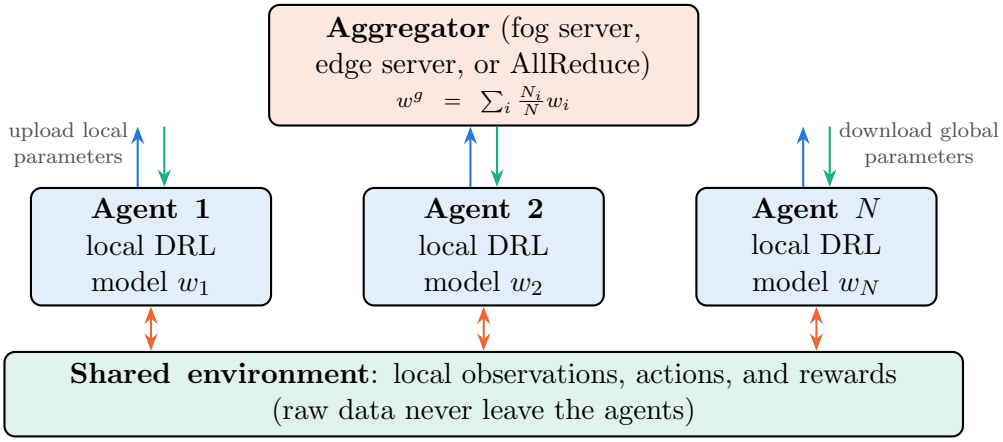


Figure 2.6.1: Federated multi-agent reinforcement learning: agents learn locally from their own interaction and periodically fuse model parameters through an aggregator

2.7 Privacy and Security in Federated Learning

Keeping data local does not by itself guarantee privacy: **gradient inversion** and inference attacks can reconstruct samples or attributes from individual updates [2]. Analyses distinguish a **semi-honest** adversary, which follows the protocol but learns what it can, from a **malicious** (Byzantine) one, which may send arbitrary updates. **Secure aggregation** ensures that the server learns only the *sum* of updates. In the pairwise-masking construction adopted in [2], each pair of agents $i < j$ derives a shared secret by elliptic-curve Diffie–Hellman (ECDH) key exchange and expands it into a pseudorandom mask m_{ij}; agent i uploads $\tilde{w}_i = w_i + M_i$ with

$$M_i = \sum_{j>i} m_{ij} - \sum_{j<i} m_{ji}. \tag{2.6}$$

Every mask appears once with each sign, so $\sum_i M_i = 0$ and the average of masked updates equals the true average (Figure 2.7.1).

Differential privacy (DP) gives a statistical guarantee: a mechanism $\mathcal{M}$ is (ϵ, δ)-differentially private if, for datasets D, D' differing in one record and any output set S, $\Pr[\mathcal{M}(D) \in S] \leq e^{\epsilon} \Pr[\mathcal{M}(D') \in S] + \delta$. It is achieved by clipping updates and adding noise; paper [4] evaluates Laplace-noise DP with $\epsilon = 0.5$, $\delta = 10^{-5}$. Malicious participants can instead mount **data or model poisoning** and **backdoor** attacks, and multi-agent RL systems can also be attacked by observation spoofing and reward manipulation [3].

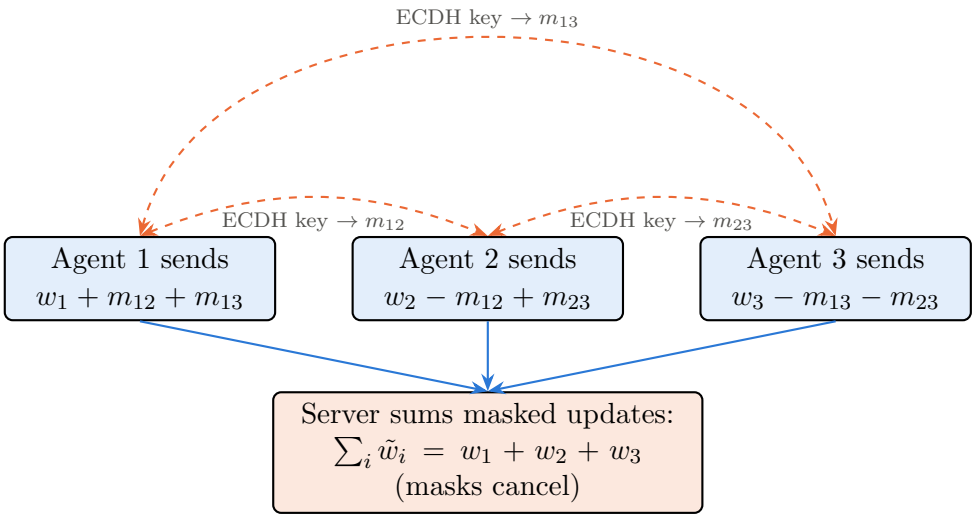


Figure 2.7.1: Secure aggregation with pairwise masks: each individual upload is hidden, but the masks cancel in the sum

Defences include robust aggregation, anomaly detection, and trust scores—but secure aggregation, by hiding individual updates, also hides the evidence needed to detect poisoned ones.

2.8 Edge Computing and Wireless Context

Mobile edge computing (MEC) places compute at base stations so devices can offload computation with low latency. In **vehicular fog computing**, tasks from moving vehicles can run locally, on neighbouring vehicles (V2V), on parked vehicles (V2P), or on roadside fog servers [1]. The offloading decision balances transmission delay, bounded by the link capacity $C = B\log_2(1+P_t\, l/N_0)$ for bandwidth B, power P_t, path-loss factor l, and noise N_0, against queuing and computation delay at the destination. With Poisson arrivals at rate λ and service rate μ, queuing delay grows sharply as λ/μ approaches one, so a task's **residence time** is highly sensitive to load [1]. In 6G, URLLC, eMBB, and mMTC tasks compete through the **Medium Access Control** (MAC) layer, and offloading, channel selection, and CPU speed interact, motivating **cross-layer** optimisation [2]. 6G will also include **UAV-enabled** networks, in which trajectory becomes a control variable coupled with power and multiple access, and **integrated sensing and communication** (ISAC) [3]. Table 2.1 lists the three 6G service classes with the task sizes and deadlines used to simulate them in [2].

Table 2.1: 6G service classes and the task profiles used to simulate them in [2]

Class	Characteristics and example applications	Task size	Deadline
URLLC	Ultra-reliable, very low latency; autonomous driving, industrial control	1 MB	2 s
eMBB	High data rate; video streaming, augmented and virtual reality	3 MB	5 s
mMTC	Many low-power devices with small, delay-tolerant messages; IoT sensing	0.5 MB	10 s

2.9 Evaluation Metrics

Table 2.2 summarises the metrics used by the reviewed works. Fairness is measured with **Jain's index** over per-agent allocations x_i,

$$F_{\text{Jain}} = \frac{\left(\sum_{i=1}^{N} x_i\right)^2}{N \sum_{i=1}^{N} x_i^2} \in \left[\frac{1}{N}, 1\right], \tag{2.7}$$

which equals 1 for equal allocations and $1/N$ when one agent receives everything.

Table 2.2: Evaluation metrics used in the reviewed works

Metric	Definition	Used in
Residence time	Time a task spends queued and executing	[1]
End-to-end delay	Transmission + queuing + computation delay	[1]
Delivery / utilisation ratio	Results returned per task offloaded; tasks computed per task received	[1]
Reliability	Rolling fraction of tasks completed within their deadline	[2]
Energy per task; energy efficiency	Joules per task; throughput per joule (bits/J)	[2]
Spectral efficiency	Throughput per unit bandwidth (bps/Hz)	[2]
Fairness	Jain's index, optionally combined with channel-access entropy	[2]
Scalability	Performance retained as the number of agents grows	[2, 4]
Training time to target accuracy	Wall-clock time until the global model reaches a set accuracy	[4]

2.10 Taxonomy of Approaches

Following the survey [3], federated multi-agent learning can be classified by its learning formulation, training topology, agent model, and application domain. Figure 2.10.1 shows

this taxonomy and the complementary positions of the four reviewed works.

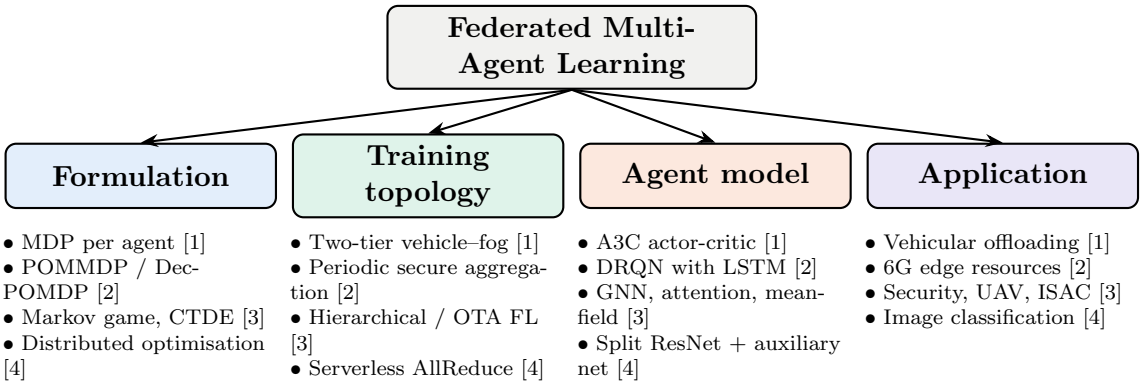


Figure 2.10.1: A taxonomy of federated multi-agent learning and the positions of the reviewed works

The taxonomy shows that the works are complementary rather than competing. The two reinforcement learning frameworks [1, 2] differ mainly in agent model and coordination: an online actor-critic with asynchronous two-tier aggregation for fast-changing vehicular networks, versus a recurrent value-based agent with periodic secure aggregation for partially observable 6G edge networks. ComDML [4] addresses the training process that both depend on, and the survey [3] places all three within a wider space of formulations, architectures, and applications that includes network security and UAV control.

2.11 Chapter Summary

This chapter introduced the MDP formulation of RL and its deep (DQN, DRQN, actor-critic, A3C) and multi-agent (Markov game, Dec-POMDP, CTDE) extensions; federated, decentralised, and split learning and their combination with RL; secure aggregation, differential privacy, and poisoning threats; the vehicular, 6G, and UAV settings; and the evaluation metrics. The next chapter uses these concepts to review the four selected works.

Chapter 3

Literature Survey

This chapter reviews the four selected works. For each, the problem, formulation, algorithm, experimental setup, and principal results are presented, followed by a brief critical discussion. The base paper [1] is reviewed first, followed by FERMI-6G [2], the survey [3], and ComDML [4].

3.1 A Federated Multi-Agent Deep Reinforcement Learning for Vehicular Fog Computing

Shabir, Rahman, Malik, Buyya, and Khan [1], in *The Journal of Supercomputing*, propose a two-tier federated multi-agent DRL framework that learns task-offloading decisions in vehicular fog computing: vehicles train local actor-critic models, and a federation of fog servers aggregates them into a global model.

3.1.1 Problem Statement and Motivation

Vehicular fog computing lets moving vehicles use the idle resources of neighbouring vehicles, parked vehicles, and roadside fog servers. Choosing *where* each task runs is hard because topology, arrival rates, and queue states change continuously [1]. Heuristics, game theory, bandits, and genetic algorithms consider few parameters or need centralised management and degrade under heavy load, while single-agent or centralised DRL converges slowly to local optima because each learner sees little of the environment. Sharing raw state is also costly and privacy-invasive. The paper asks whether federating the learning—training on each vehicle and sharing only parameters with the fog tier—gives faster convergence and better offloading with less communication and more privacy.

3.1.2 System Model and Problem Formulation

The environment contains moving vehicles $\mathcal{V}$, parked vehicles $\mathcal{P}$, and fog servers $\mathcal{F}$; only moving vehicles generate tasks. The path loss at distance d is $l = a + \beta \log_{10}(d) + b$ (offset a, shadow fading b, attenuation index β), and the channel is assumed quasi-static and homogeneous. Each task j has size s_j (MB) and computing requirement c_j (Mbits). Nodes are multi-server FIFO queues with Poisson arrivals at rate λ, and the **residence time** of agent i with clock frequency f_i and queue Q is

$$Z_i = \frac{\sum_{x \in Q} c_x}{f_i}. \tag{3.1}$$

The service time of a task is $D_i = D_i^1 + D_i^2 + D_i^3$, the sum of the transmission delay $D_i^1 = \psi_i B(D_i^{\uparrow} + D_i^{\downarrow})$ weighted by the frequency-reuse cost ψ_i, the computation delay $D_i^2 = (\psi_2 f_i) \sum_j c_j / f_i$ with unit lease cost ψ_2, and the queuing delay $D_i^3 = (\psi_2 f_i) \sum_j Z_j$ [1].

Each vehicle is an agent with state

$$S_i = \{Z_i,\ \Delta_i,\ V_i,\ W_i,\ f_i,\ (s_j, c_j)\}, \tag{3.2}$$

containing the residence times of, distances to, relative speeds of, and offloading modes of the available resources, the agent's own capacity, and the task attributes. The action space $A_i = \{a_1, a_2, a_3, a_4\}$ executes the task locally or offloads it to a moving vehicle, a fog server, or a parked vehicle. The reward for executing at destination k is

$$r_i = -\frac{1}{\lambda} Z_k, \tag{3.3}$$

so destinations with shorter queues earn more reward. The objective is to minimise the expected discounted value, which corresponds to task waiting time, subject to binary decisions and workload not exceeding capacity [1].

3.1.3 Multi-Agent A3C and Two-Tier Federation

Each agent learns with A3C (Section 2.2). With advantage $A(s_t, a_i; \theta, w) = r_t + \gamma V(s_{t+1}; w) - V(s_t; w)$, the actor and critic gradients are

$$d\theta \leftarrow d\theta + \nabla_\theta \log \pi_\theta(a_i \mid s_t)\, A + \beta \nabla_\theta H\big(\pi(s_t;\theta)\big), \qquad dw \leftarrow dw + \frac{\partial A^2}{\partial w}, \tag{3.4}$$

where the entropy term prevents premature convergence [1]. Learning is **online**: parallel environments generate fresh experience, which suits a setting for which no complete dataset exists.

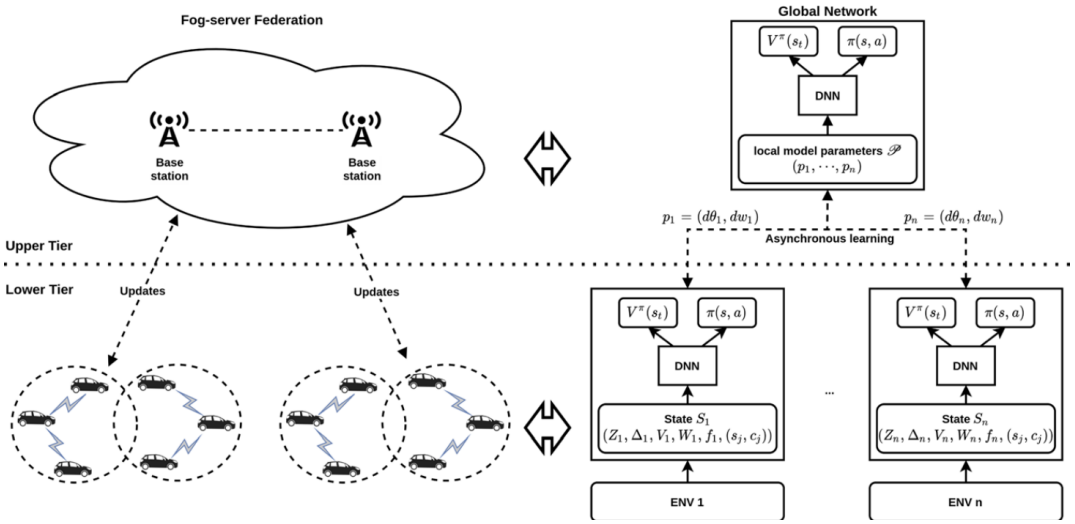


Figure 3.1.1: Two-tier federated learning framework: vehicles in the lower tier train local actor-critic models, and the fog-server federation in the upper tier maintains the global network [1]

Figure 3.1.1 shows the architecture. In the **lower tier**, each vehicle initialises its local actor-critic from the global network, learns online for up to $t_{\max}$ steps, and computes policy and value gradients. In the **upper tier**, local weights are sent after every episode to a federation of fog servers; one under-loaded server, $\arg\min_{i \in \mathcal{F}} Z_i$, is delegated to maintain the global network, which aggregates the local models asynchronously and returns the result (Algorithm 2). The fog tier connects to many more vehicles than any single vehicle does, so it gives each agent a wider view of the environment, while only model parameters cross the network [1].

Algorithm 2 Two-tier federated A3C task offloading (summarised from [1])

Require: vehicles $\mathcal{V}$ with local networks (θ_i, w_i); fog federation $\mathcal{F}$; episode length $t_{\max}$

Delegate fog server $f^* \leftarrow \arg\min_{f \in \mathcal{F}} Z_f$ to host the global network (θ, w)
for each episode **do**
  for each vehicle $i \in \mathcal{V}$ **in parallel do**
    $(\theta_i, w_i) \leftarrow (\theta, w)$
    for $t = 1$ to $t_{\max}$ **do**
      Build S_i on task arrival; choose $a_i \sim \pi(a \mid S_i; \theta_i)$ among local, vehicle, fog, parked
      Execute at destination k; observe reward $r_i = -Z_k/\lambda$
    end for
    Compute advantage, policy loss with entropy, and value loss; send $(d\theta_i, dw_i)$ to $\mathcal{F}$
  end for
  f^* aggregates updates asynchronously into (θ, w) and returns the global model
end for

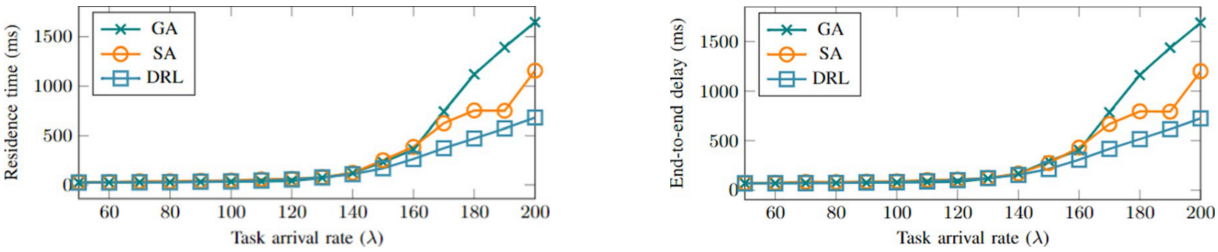
3.1.4 Experimental Setup

The framework is simulated in SUMO on a Manhattan grid of 900 m × 900 m for 1000 time steps with 100 moving vehicles, 20 parked vehicles, and 9 fog servers. Vehicle and fog clocks are 0.6–1.0 GHz and 2.0 GHz, transmission ranges 100 m and 200 m, and V2V, V2R, and V2P bandwidths 1, 2, and 1 MHz; tasks need 0.2–1.0 Mbits of computation and carry 1–6 MB of data. The actor-critic network has layers of 32, 16, 8, and 4 neurons with tanh and softmax activations and the Adam optimiser (learning rate 10e−5, discount 0.955), implemented in TensorFlow 1.13.1 with two parallel simulation threads. Arrival rates range from $\lambda = 60$ to 200 tasks/s. Baselines are the **Stochastic Selection Algorithm** (SA), which picks a destination uniformly at random within range, and the **Greedy Algorithm** (GA), which picks the nearest node [1].

3.1.5 Results and Analysis

Delay and cost. At low load all methods behave alike, but beyond $\lambda = 140$ GA and SA overload some nodes while the DRL scheme spreads the load (Figure 3.1.2). At $\lambda = 200$ it reduces residence time by **58.6%** and end-to-end delay by **57.11%**, while SA and GA incur 16% and 56% higher system cost [1]. GA has the lowest *transmission* delay because it always offloads to the nearest node, which is exactly what creates its long queues; the DRL agent accepts longer transmissions when a farther node has a much

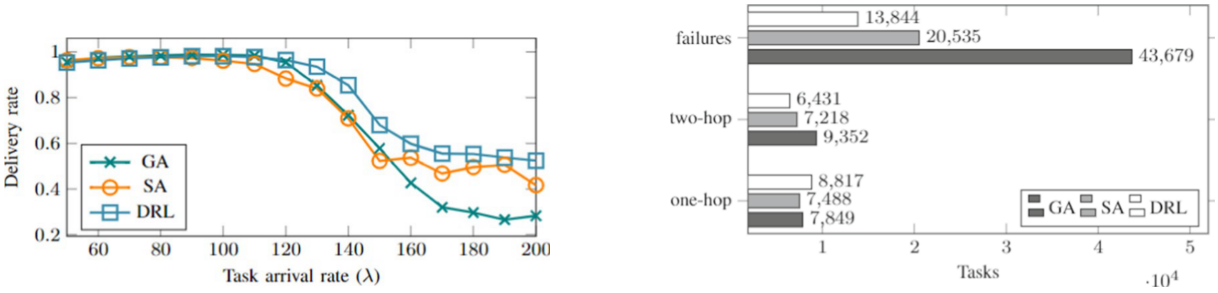
shorter queue.



(a) Mean residence time **(b)** Mean end-to-end delay

Figure 3.1.2: Residence time and end-to-end delay versus task arrival rate for GA, SA, and the proposed DRL scheme [1]

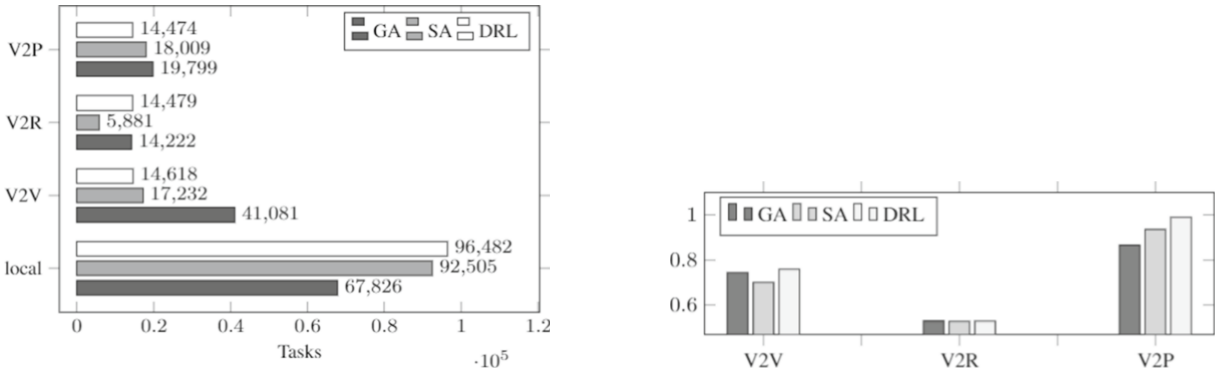
Delivery and failures. The delivery rate of GA and SA drops by 45.9% and 20.4% at $\lambda = 200$, while the DRL scheme stays highest (Figure 3.1.3). It delivers about 15,000 tasks, 8,817 of them in a single hop—the most first-hop deliveries of the three methods—and records 13,844 failures against 20,535 for SA and 43,679 for GA, a reduction the authors summarise as 69% relative to GA [1].



(a) Result delivery ratio **(b)** Task delivery statistics at $\lambda = 200$

Figure 3.1.3: Delivery ratio versus arrival rate, and one-hop deliveries, two-hop deliveries, and failures at $\lambda = 200$ [1]

Workload and utilisation. The DRL scheme executes the most tasks locally (96,482 against 92,505 for SA and 67,826 for GA, reported as a 30% gain over GA) and spreads the rest almost evenly, about 14,500 each, over moving vehicles, fog servers, and parked vehicles, whereas GA sends 41,081 tasks to neighbouring vehicles (Figure 3.1.4). It achieves the highest utilisation ratio, improving V2P by 5.35% and V2V by 8.5%. Actor and critic losses settle after about 10^4 iterations, confirming convergence [1].



(a) Tasks allocated per destination **(b)** Utilisation ratio

Figure 3.1.4: Workload distribution and utilisation ratio at $\lambda = 200$ tasks/s [1]

3.1.6 Discussion

The paper combines multi-agent DRL, asynchronous A3C, and two-tier vehicle–fog federation in a practical framework with large gains under heavy load, and its residence-time reward is simple and well aligned with the goal of avoiding congested nodes. Its limitations are that the baselines are simple heuristics, the channel is assumed homogeneous and quasi-static, only two training threads were used, and—as the authors acknowledge—updates are shared in plain form, leaving the system open to gradient spoofing and inference attacks. These gaps are addressed by the secure aggregation of FERMI-6G [2] and the communication-efficient training of ComDML [4].

3.2 Federated Multi-Agent Reinforcement Learning for Privacy-Preserving and Energy-Aware Resource Management in 6G Edge Networks

Andong and Min [2] propose **FERMI-6G** (Federated Energy-aware Reinforcement for Multi-agent Intelligence in 6G), in which edge agents jointly decide task offloading, spectrum access, and CPU frequency and synchronise through cryptographic secure aggregation.

3.2.1 Problem Statement and Motivation

Ultra-dense, energy-constrained 6G networks must serve URLLC, eMBB, and mMTC traffic. Centralised resource management suffers from scalability bottlenecks, signalling overhead, and privacy risk, existing MARL solutions often assume full observability and

optimise one metric at one layer, and standard FL is vulnerable to gradient inversion [2]. FERMI-6G aims to be decentralised, private, energy-aware, and cross-layer under partial observability and mobility.

3.2.2 System Model and Reward

At each interval, agent i selects a composite **cross-layer action**

$$a_i(t) = \big(a_i^{\text{app}}(t),\ a_i^{\text{mac}}(t),\ a_i^{\text{cpu}}(t)\big), \tag{3.5}$$

choosing local execution or offloading ($a_i^{\text{app}} \in \{0, 1\}$), a channel ($a_i^{\text{mac}} \in \{1, \ldots, k\}$), and a CPU frequency ($a_i^{\text{cpu}} \in \mathbb{R}$). The environment is a POMMDP in which agent i observes only $o_i(t) = \{q_i, E_i, h_i, \lambda_i, c_i\}$: queue, residual energy, channel gain, task type, and CPU usage [2]. Each agent uses a DRQN with prioritised replay (Figure 3.2.1).

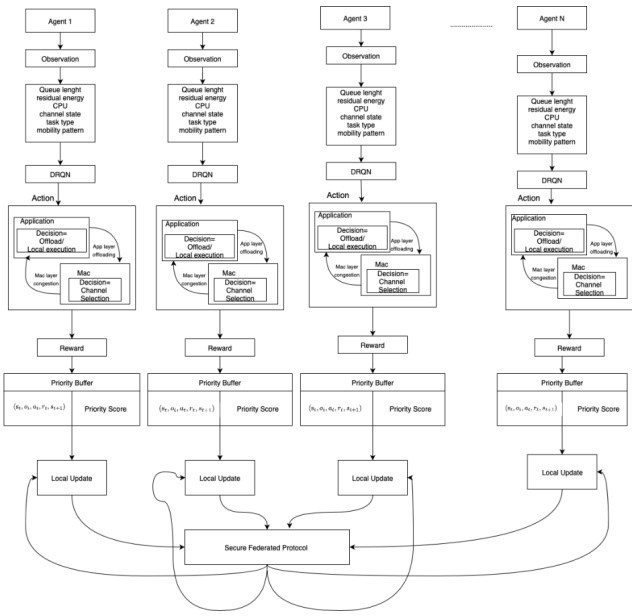


Figure 3.2.1: System architecture of FERMI-6G: each agent observes local state, selects application- and MAC-layer actions with a DRQN, stores prioritised experience, updates locally, and joins the secure federated protocol [2]

The reward combines normalised latency $\tilde{L}_i = L_i/D_i$ (latency over deadline), normalised energy per task $\tilde{E}_i = E_i/E_{\max}$, a **hybrid fairness** index $F_i = \beta_{\text{jain}} F_{\text{Jain}} + \beta_{\text{entropy}} H_{\text{avg}}$ that mixes Jain's index (Equation 2.7) with the average normalised channel-access entropy, reliability R_i (a rolling task-success average), spectral efficiency, energy

efficiency, and MAC success rate:

$$r_i(t) = w_L \tilde{L}_i^{-1} P_{\text{dyn}} + w_E \tilde{E}_i^{-1} P_{\text{energy}} + w_F F_i + w_R R_i + w_{SE} \widetilde{SE}_i + w_{EE} EE_i + w_{MAC}\, \text{MAC}_{\text{rate},i}, \tag{3.6}$$

where the penalties P_{dyn} and P_{energy} double the latency and energy terms when latency exceeds 2 s or energy falls below a threshold [2]. The reward is also decomposed into application-layer (latency and computation energy) and MAC-layer (transmission energy and fairness) components so that agents learn how the two layers interact.

3.2.3 Secure Aggregation and the FERMI-6G Algorithm

Every K episodes, agents synchronise with the pairwise-masking protocol of Section 2.7, adapted from Google's secure aggregation framework [7] to federated MARL: masks derived from ECDH keys and AES cancel in the sum, so the aggregator recovers the exact average without seeing any individual model under a semi-honest adversary. Only agents whose energy exceeds $E_{\text{threshold}}$ take part in a round [2]. The global objective maximises long-term reward subject to minimum energy per agent, partial observability, and a communication budget. FERMI-6G's four innovations are temporal DRL with LSTM-based DRQNs, the cross-layer action space, secure synchronisation, and optional adaptive reward weights driven by moving averages of system metrics. Algorithm 3 condenses the method.

3.2.4 Experimental Setup

Agents move by random walk (0.1–1.0 m/s) on a 100×100 grid with the edge server at $[50, 50]$. Tasks are URLLC (1 MB, 2 s deadline), eMBB (3 MB, 5 s), or mMTC (0.5 MB, 10 s); batteries must stay above 20%; the MAC layer models congestion, collisions, and retransmissions (probability 30%). Training runs 800 episodes of 20 steps with 5 agents, a 10,000-transition buffer, batch size 16, $\gamma = 0.99$, learning rate 10^{-3}, and sequence length 20 on a 2016 laptop [2]. Four frameworks are compared: **FERMI-6G**; **Fed-MARL**, the same federated framework without cross-layer learning (its MAC and CPU decisions are made by heuristics, which raised its reliability from about 30% to about 94%); **Centralised RL** with a single-layer policy; and **Centralised RL (Cross-Layer)**.

Algorithm 3 FERMI-6G local learning and secure aggregation (condensed from [2])

Require: agents $\mathcal{A}$ with DRQN θ_i, target θ_i^{target}, replay B_i, ECDH keys; episodes N; steps T; interval K

for episode $n = 1$ to N **do**
  for $t = 1$ to T, for each agent i **do**
    $h_i \leftarrow \text{LSTM}(h_i, o_i(t))$; select $a_i = (a_i^{\text{app}}, a_i^{\text{mac}}, a_i^{\text{cpu}})$ ϵ-greedily from $Q(h_i, \cdot\,; \theta_i)$
    Execute; store $(h_i, a_i, r_i, o_i(t+1))$ with priority; update θ_i on a prioritised mini-batch
  end for
  if $n \bmod K = 0$ **then**
    Each agent with $E_i > E_{\text{threshold}}$ sends $\theta_i + M_i$ with $M_i = \sum_{j>i} m_{ij} - \sum_{j<i} m_{ji}$
    Aggregator broadcasts $\theta_g = \frac{1}{|\mathcal{A}|} \sum_i (\theta_i + M_i)$; agents set $\theta_i, \theta_i^{\text{target}} \leftarrow \theta_g$
    (Optional) adapt and renormalise reward weights
  end if
end for

Because the centralised frameworks obtained rewards near -147 and zero reliability in the full environment, they were evaluated in simplified environments.

3.2.5 Results and Analysis

Table 3.1 reports the final performance. FERMI-6G has the highest reward (59.83), reliability (**96.83%**), energy efficiency (**68.72 bits/J**), throughput (29.20 Gbps), and the lowest energy per task (**0.0230 J**) and failure rate (3.17%) [2].

Table 3.1: Framework comparison, mean ± standard deviation [2]

Metric	FERMI-6G	Central. RL	Central. RL (Cross-Layer)	Fed-MARL
Reward	**59.83 ± 6.70**	51.58 ± 2.42	53.56 ± 2.97	51.51 ± 8.03
Reliability (%)	**96.83 ± 4.21**	89.78 ± 5.32	93.81 ± 6.13	94.00 ± 7.31
Energy eff. (bits/J)	**68.72 ± 19.99**	19.55 ± 0.71	19.84 ± 0.74	57.80 ± 16.37
Energy (J/task)	**0.0230 ± 0.0075**	0.0663 ± 0.0038	0.0683 ± 0.0040	0.0264 ± 0.0072
Spectral eff. (bps/Hz)	0.1981 ± 0.3750	0.5391 ± 0.0511	**0.5642 ± 0.0575**	0.2189 ± 0.4485
Latency (s)	1.12 ± 0.69	1.36 ± 0.09	**0.92 ± 0.40**	1.11 ± 0.73
Completion time (s)	0.95 ± 0.52	1.32 ± 0.63	**0.72 ± 0.23**	0.92 ± 0.53
Failure rate (%)	**3.17 ± 4.21**	10.22 ± 5.32	6.19 ± 6.13	5.99 ± 7.31
Privacy protection	**High**	Low	Low	**High**
Offloading delay (s)	3.95 ± 0.79	0.88 ± 0.76	**0.70 ± 0.63**	3.91 ± 0.78
Throughput (Gbps)	**29.20 ± 2.75**	11.35 ± 0.13	14.11 ± 0.14	28.26 ± 2.88
Reliab., 50 agents (%)	**90.01 ± 0.60**	12.38 ± 4.92	20.69 ± 9.10	84.05 ± 5.17
Fairness (0–1)	0.79 ± 0.07	0.76 ± 0.05	**0.99 ± 0.0046**	0.17 ± 0.07

Learning dynamics. FERMI-6G's reliability stays flat for about 100 episodes while the cross-layer policy is learned, then rises to 96.83% by episode 800; the centralised cross-layer framework, helped by its global view, improves immediately and settles at 93.81% by episode 400, and Fed-MARL fluctuates between 30% and 70% until episode 300 (Figure 3.2.2). FERMI-6G's energy per task falls from about 0.035 J to 0.011 J over training, a 70% reduction, as it learns to avoid redundant computation, whereas the centralised frameworks show no downward trend [2].

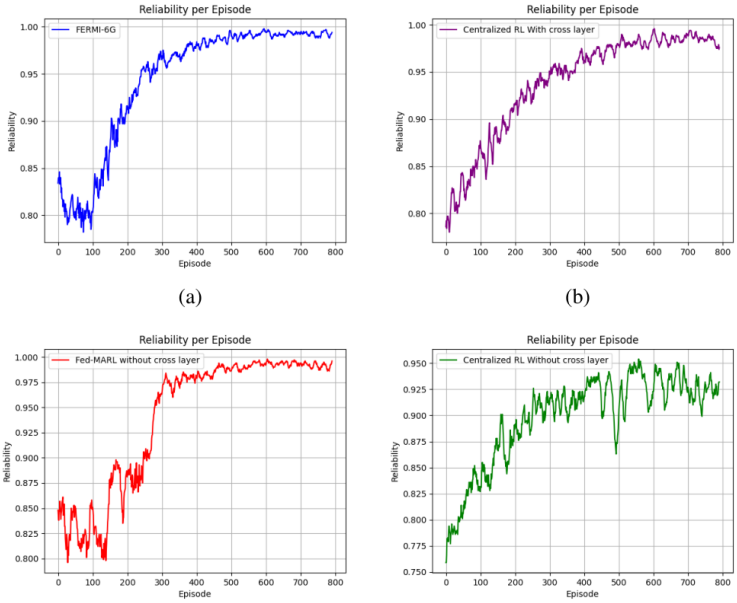
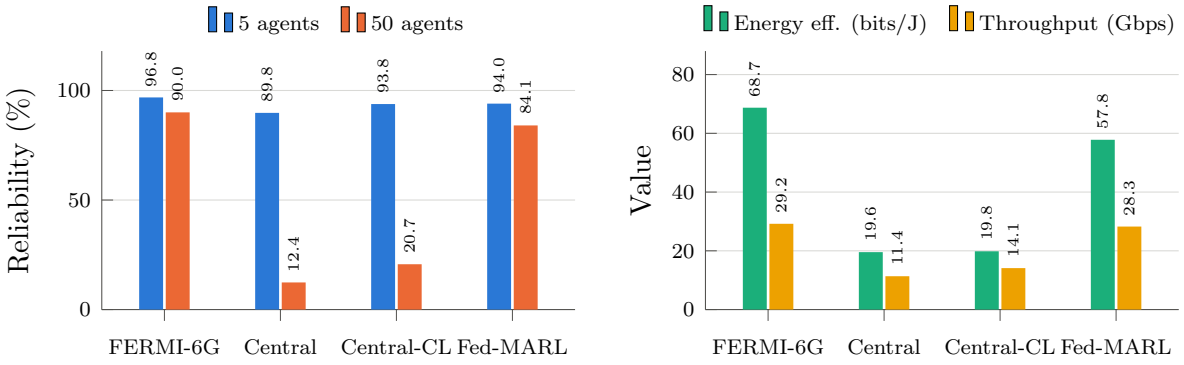


Figure 3.2.2: Reliability per episode: (a) FERMI-6G, (b) centralised RL with cross-layer optimisation, (c) Fed-MARL without cross-layer optimisation, (d) centralised RL [2]

Scalability. With 50 agents, FERMI-6G retains 90.01% reliability and Fed-MARL 84.05%, whereas the centralised frameworks collapse to 12.38% and 20.69% (Figure 3.2.3). The centralised cross-layer framework does achieve the lowest latency (0.92 s) and completion time, but in its simplified environment [2].

3.2.6 Discussion

FERMI-6G shows that decentralised agents can learn a genuinely cross-layer policy while protecting their updates cryptographically, and that this scales far better than centralised control. The authors report its trade-offs candidly. Performance gains make agents more "selfish": their discussion notes fairness falling to about 0.37 against about



(a) Reliability with 5 and 50 agents

(b) Energy efficiency and throughput

Figure 3.2.3: Scalability, energy efficiency, and throughput of the four frameworks, plotted from Table 3.1 (Central-CL: centralised RL with cross-layer optimisation)

0.80 for the centralised cross-layer framework (Table 3.1 reports 0.79 and 0.99). Offloading delay (3.95 s) is high and spectral efficiency (about 0.20 bps/Hz) is far below 6G targets of 10–20 bps/Hz. The baselines ran in simplified environments, so numerical comparisons partly reflect environment differences; the main experiment uses only five agents; synchronisation every K episodes is assumed; and the threat model covers only semi-honest adversaries, so poisoned updates from malicious agents would go undetected. Fed-MARL with heuristic MAC and CPU control is simpler and easier to tune, but in dynamic, congested networks FERMI-6G's learned cross-layer policy performs better.

3.3 Federated Multi-Agent Deep Learning and Neural Networks for Advanced Distributed Sensing in Wireless Networks

Muller, DeRosa, Zhang, and Huan [3], from the University of Oulu, survey multi-agent deep learning (MADL)—multi-agent DRL, federated training, and graph neural networks—for wireless systems in which sensing, communication, and computing are tightly coupled, with emphasis on 2021–2025 research. It is the broadest of the four works and the main source in this seminar for the network-security and UAV aspects of federated multi-agent AI.

3.3.1 Scope, Background, and Reference Architecture

Integrated sensing and communication (ISAC), edge intelligence, open programmable RAN (O-RAN), and UAV networking create control problems that are decentralised,

partially observed, time-varying, and resource-constrained [3]. The survey contributes a systems-to-algorithms **taxonomy**, a synthesis of **advanced techniques** (hierarchical and over-the-air FL, communication-efficient federated deep RL, serverless edge computing), coverage of **applications** (MEC offloading and slicing, UAV networks with power-domain NOMA, intrusion detection, ISAC), and an **open-issues agenda**. It models wireless problems as Markov games or Dec-POMDPs trained with CTDE, uses graph neural networks to exploit the permutation symmetry of interference graphs, and places learning agents at edge nodes that must act in real time. Figure 3.3.1 redraws its three-plane reference architecture.

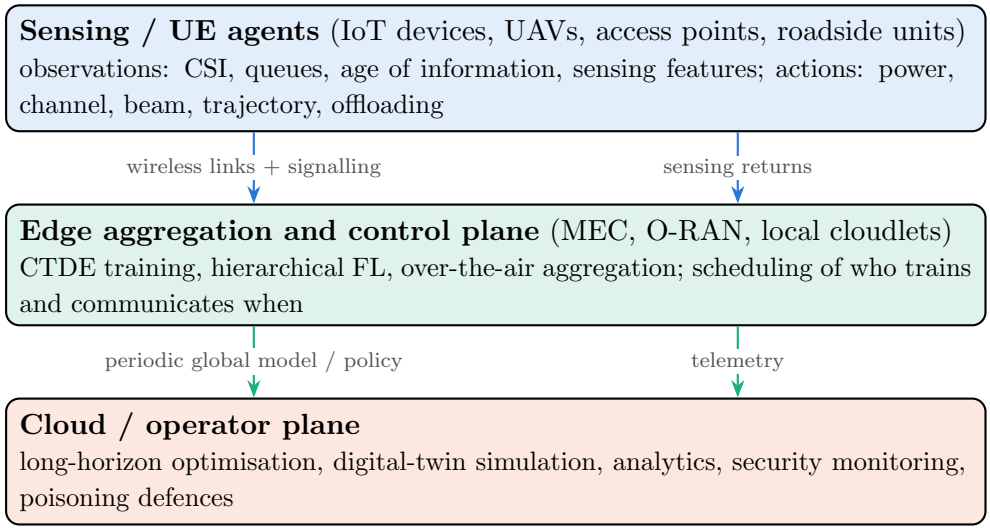


Figure 3.3.1: Conceptual architecture for distributed sensing, wireless communication, and multi-agent deep learning (redrawn from [3])

3.3.2 Advanced Techniques

Multi-agent DRL casts each base station, user, UAV, or slice controller as an agent, with **mean-field** approximations for dense networks and **constrained** formulations where QoS is non-negotiable. **GNN-based learning-to-optimise** generalises across network sizes through permutation equivariance. **Wireless FL** must be co-designed with client selection and power and bandwidth allocation; hierarchical FL reduces latency, and over-the-air aggregation trades speed for fading-induced error. **Federated deep RL** has been applied to cooperative edge caching and aerial relays—the vehicular and 6G frameworks of [1, 2] belong here. **Serverless edge computing** can package inference, aggregation, and

monitoring as on-demand functions, at the risk of cold starts [3]. Table 3.2 summarises the survey's comparison.

Table 3.2: Core learning paradigms and their suitability for wireless systems (adapted from [3])

Paradigm	**Key strength**	**Key limitation**
CTDE MADRL	Captures coupling such as interference with low run-time signalling	Non-stationarity; scaling; safety and QoS constraints
Mean-field MADRL	Scales to dense deployments	May miss rare but critical interactions
GNN learning-to-optimise	Permutation equivariance; generalises across topologies	Graph and feature design; missing CSI
Wireless / hierarchical FL	Privacy by design; lower-latency aggregation	Communication bottleneck; heterogeneity; security threats
Federated deep RL	Distributed adaptation without sharing trajectories	Instability; partial observability; poisoning risks
Serverless orchestration	Elasticity; fits intermittent learning tasks	Cold starts; orchestration complexity

3.3.3 Applications: UAV Networks, Sensing, and Security

For **MEC and slicing**, the survey cites slicing-aware offloading with an adaptive TD3 agent and constrained multi-agent RL for RAN slicing in O-RAN. For **UAV networks**, multi-agent DRL has optimised UAV **trajectories** for differentiated services, and **federated multi-agent DRL** has achieved fair communication and trajectory control across multiple UAVs while keeping each UAV's experience local; other work combines serverless FL with power-domain NOMA in UAV-enabled heterogeneous networks, and federated DRL jointly optimises UAV trajectory and beamforming in space–aerial–terrestrial relay networks with RIS and RSMA [3]. For **ISAC**, agents allocate beams and waveforms between sensing and communication and fuse detections across nodes for tracking. For **security**, intrusion detection in sensor networks increasingly uses FL for privacy and cross-domain training, but collaboratively trained detectors are exposed to **poisoning and backdoor** attacks, bridging "sensing security" and "learning security" [3].

3.3.4 Challenges and Future Directions

The survey identifies six challenges: **scalability and non-stationarity** in dense networks and swarms; **communication overhead**, since model updates compete with user

traffic; **real-time safety**, where formal guarantees for deep RL are lacking; **adversarial security**, including poisoning, backdoors, observation spoofing, reward manipulation, and Byzantine agents; **heterogeneity and reproducibility**, given custom simulators and proprietary traces; and **operational complexity** of O-RAN control loops and serverless deployment. It proposes sense–communicate–compute co-design with hybrid model-based and learned controllers, constraint-aware and verifiable policies, graph-native multi-agent learning, federated RL at scale with robust aggregation, serverless learning control planes, and privacy-aware real-time ISAC intelligence [3].

3.3.5 Discussion

The survey connects the other works to a larger map—[1, 2] are federated deep RL for MEC, FERMI-6G's secure aggregation addresses one of its security concerns, and ComDML [4] answers its call for communication-efficient serverless learning—and it is the only work here to cover UAV trajectory control, intrusion detection, and ISAC. As a short survey, however, it contains no experiments, its comparisons are qualitative, and some in-text citation numbers do not match its reference list (for example, a citation for GNN-based radio resource management points to an unrelated routing paper), so original sources should be consulted directly.

3.4 Communication-Efficient Training Workload Balancing for Decentralized Multi-Agent Learning

Mohammadabadi, Yang, Yan, and Zhang [4] propose **ComDML**, a decentralised multi-agent learning (DML) framework that balances training workload among heterogeneous agents without any central server: slow agents offload part of their model to fast agents using local-loss-based split training, and a decentralised scheduler decides the pairings.

3.4.1 Problem Statement and Motivation

Devices differ in computation, bandwidth, and data size, so in synchronous training fast agents wait for **stragglers** and their spare capacity is wasted [4]. Split learning reduces device computation but waits for server gradients at every step; fixed-split and tier-based methods cannot adapt or still train the whole model on each device; and all of these need

a server, which is a bottleneck and a target. Serverless methods such as gossip learning and BrainTorrent remove the server but ignore heterogeneity. ComDML minimises total training time in a heterogeneous, fully decentralised system.

3.4.2 Local-Loss Split Training and Optimisation

The K agents minimise the global objective of Equation 2.5. The model is split at one of M points m into a **slow agent-side** part $w_{s_i}^{a_m}$ and a **fast agent-side** part $w_{f_i}^{a_m}$. The slow agent trains its part with a small auxiliary network (a fully connected and an average-pooling layer) that provides a local loss, while the paired fast agent trains the remaining layers on the slow agent's intermediate outputs z_n—in parallel with its own training:

$$\min_{w_{s_i}^{a_m},\, w_i^{\text{aux}_m}} \frac{1}{N_i}\sum_{n=1}^{N_i} \ell\big((x_n, y_n); (w_{s_i}^{a_m}, w_i^{\text{aux}_m})\big), \qquad \min_{w_{f_i}^{a_m}} \frac{1}{N_i}\sum_{n=1}^{N_i} \ell\big((z_n, y_n); w_{f_i}^{a_m}\big). \tag{3.7}$$

Figure 3.4.1 shows how offloading fills the fast agent's idle time [4].

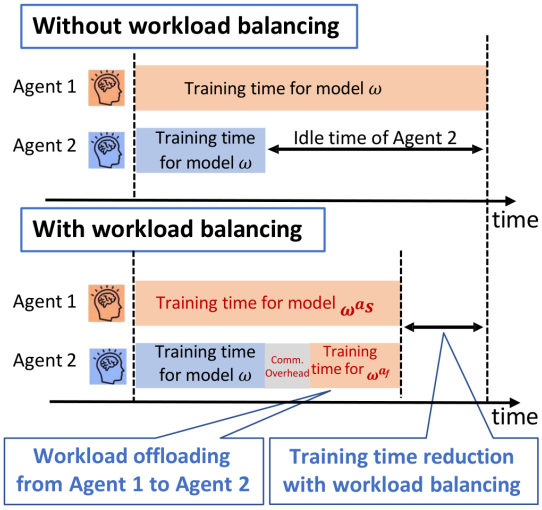


Figure 3.4.1: Training with and without workload balancing: offloading part of Agent 1's model to Agent 2 fills Agent 2's idle time and shortens the round [4]

With $\gamma_{ij} \in \{0, 1\}$ indicating that agent i offloads to agent j, an agent that does not offload spends its own computation time $\tau_i(w)$ plus the computation and communication for work accepted from slower agents, while an agent that offloads spends $\tau_i(w_{s_i}^{a_m})$ plus the

communication time τ_{ij} of its intermediate data. Since the round ends when the slowest agent finishes, ComDML solves

$$\min_{\{\gamma_{ij}\},\{w_{f_i}^{a_m}\}} \max_i \tau_i \qquad \text{s.t. } \gamma_{ij} \in \{0,1\} \;\; \forall i,j, \tag{3.8}$$

an integer program that is hard to solve exactly without a central scheduler [4].

3.4.3 Decentralised Pairing and Training Workflow

Each agent first profiles every split m: the relative training time of the slow side $T_s^{a_m}$ and fast side $T_f^{a_m}$ and the intermediate data size ν_m per batch. Each round, agents broadcast their processing speed p_j and individual training time $\hat{\tau}_j$ and pair greedily, slowest first. Agent i with $\tilde{N}_i$ batches estimates its time when offloading to j over a link of speed c_{ij} as

$$\hat{\tau}_{ij}^m = \max\Big(\frac{\tilde{N}_i}{p_i^m},\ \hat{\tau}_j + \frac{\tilde{N}_i\,\nu_m}{c_{ij}} + \frac{\tilde{N}_i}{p_j^m}\Big), \qquad p_i^m = \frac{p_i}{T_s^{a_m}},\ p_j^m = \frac{p_j}{T_f^{a_m}}, \tag{3.9}$$

and chooses the partner and split that minimise it. Paired agents then train in parallel, and all models are averaged with recursive halving-and-doubling **AllReduce**, needing $2\log_2 K$ steps and no coordinator (Algorithm 4, Figure 3.4.2) [4].

Algorithm 4 ComDML: decentralised pairing and training (adapted from [4])

Require: rounds R; split profiles $(T_s^{a_m}, T_f^{a_m}, \nu_m)$; agents sorted by training time (list $\mathcal{L}$)

1: **for** round $r = 0$ to $R-1$ **do**
2: Each agent broadcasts p_j and $\hat{\tau}_j$ to connected agents
3: **for** each unpaired agent i in order $\mathcal{L}$ (slowest first) **do**
4: Compute $\hat{\tau}_{ij}^m$ for every unpaired neighbour j and split m
5: $(j^\star, m^\star) \leftarrow \arg\min_{j,m} \hat{\tau}_{ij}^m$; pair with $j^\star$ if faster than training alone
6: **end for**
7: Pairs perform local-loss split training in parallel; unpaired agents train alone
8: Aggregate all models with decentralised AllReduce
9: **end for**

Privacy and convergence. Fast agents' data are protected because communication is one-way, and slow agents can add PixelDP noise, patch shuffling, distance-correlation minimisation, or SplitGuard to their intermediate data, with DP or cryptography for aggregation. Under standard assumptions (smoothness, bounded gradients and variance,

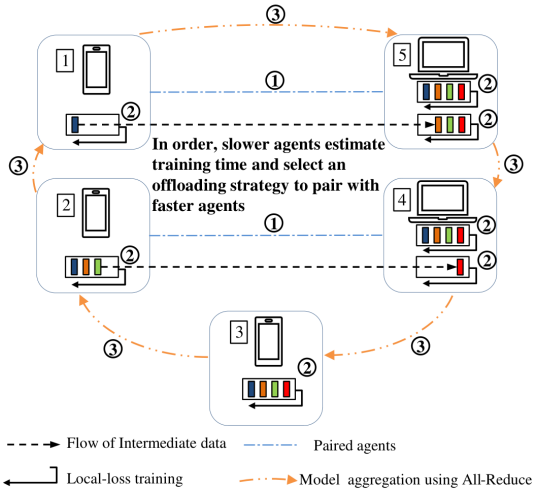


Figure 3.4.2: ComDML training round: agents 1 and 2 offload work to faster agents 5 and 4, agent 3 trains independently, and all models are aggregated with AllReduce [4]

bounded dissimilarity), the slow-side model converges at rate $\mathcal{O}\left(\frac{1}{RA_m} + e^{-R}\right)$ for strongly convex losses and $\mathcal{O}\left(\frac{1}{\sqrt{RA_m}} + \frac{1}{R}\right)$ for non-convex ones, and the fast side converges at similar rates that depend on the slow side's convergence [4].

3.4.4 Experimental Setup

Experiments train ResNet-56 and ResNet-110 models on CIFAR-10, CIFAR-100, and CINIC-10, with non-IID variants from a Dirichlet distribution (concentration 0.5), against BrainTorrent, Gossip Learning, decentralised AllReduce, and FedAvg. Agents are given one of five CPU profiles (4 to 0.2 CPUs) and five link profiles (0 to 100 Mbps), and 20% of profiles change every 100 rounds. Training uses SGD with momentum 0.9, learning rate 0.001, batch size 100, and one local epoch, in PyTorch on a server with four GTX 1080 Ti GPUs [4].

3.4.5 Results and Analysis

Choosing the split. Table 3.3 shows two-agent training to 90% accuracy. With 2 CPUs versus 0.25 CPU at 50 Mbps, offloading 37 layers cuts total time from 20,096 s to 9,352 s (**53%**) by eliminating idle time; with 2 CPUs versus 1 CPU at 100 Mbps, the best choice is 19 layers and the gain is small. Offloading more layers lowers computation but can raise communication, so the best split depends on the resources—hence dynamic pairing [4].

Table 3.3: Two-agent training time (s) to 90% accuracy on CIFAR-10 with ResNet-56 for different numbers of offloaded layers [4]

Layers	Setting 1: 2 vs 0.25 CPU, 50 Mbps				Setting 2: 2 vs 1 CPU, 100 Mbps			
	Train	Comm.	Idle	Total	Train	Comm.	Idle	Total
0	5573	34	14489	20096	5578	17	3560	9165
10	6740	3532	4787	15059	6364	1766	350	8481
19	7625	3544	1682	12851	6547	1772	137	**8456**
28	7906	1261	2049	11217	7859	630	3368	8490
37	8003	1265	84	**9352**	8275	632	7318	8908
55	10343	640	9833	10983	10101	320	9964	10421

Comparison with baselines. With 10 heterogeneous agents, ComDML is fastest on every dataset in both IID and non-IID settings (Table 3.4, Figure 3.4.3). On IID CIFAR-10 it needs 7,211 s against 24,174 s for FedAvg and 24,639 s for BrainTorrent—**70%** and **71%** less—at the same target accuracy [4].

Table 3.4: Training time (s) with 10 agents to reach the target accuracy (CIFAR-10: 90% IID / 85% non-IID; CIFAR-100: 65% / 60%; CINIC-10: 75% / 65%) [4]

Method	CIFAR-10		CIFAR-100		CINIC-10	
	IID	non-IID	IID	non-IID	IID	non-IID
ComDML	**7211**	**4177**	**5589**	**8104**	**10229**	**17208**
Gossip Learning	20337	15269	15262	28621	24636	56325
BrainTorrent	24639	14323	18046	25867	31992	51144
AllReduce	25153	13859	18462	26623	32652	53265
FedAvg	24174	13095	17630	25113	30601	49624

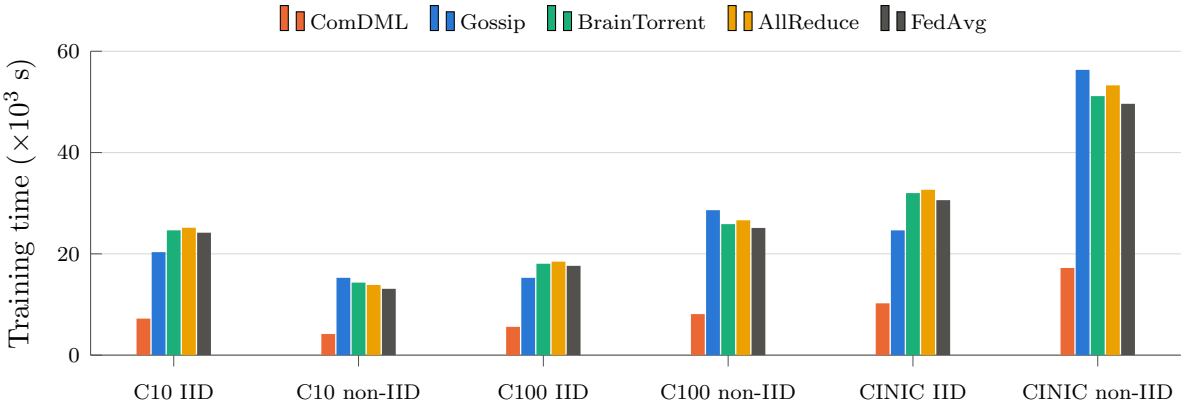


Figure 3.4.3: Training time to target accuracy with 10 heterogeneous agents, plotted from Table 3.4

Scalability, privacy, and topology. Training to 80% accuracy on IID CIFAR-10 with 20, 50, and 100 agents, ComDML stays fastest for both models; with 100 agents and ResNet-110 it needs 15,843 s against 29,494 s for FedAvg and 31,526 s for BrainTorrent. With 100 agents it reaches 81.7% accuracy with distance correlation, 83.2% with patch shuffling, and 77.6% with Laplace DP ($\epsilon = 0.5$), and it remains fastest when only 20% of links are available, because it adapts its pairings and lets poorly connected agents train alone [4].

3.4.6 Discussion

ComDML's insight is that heterogeneity is an opportunity: fast agents' idle capacity can absorb slow agents' work, provided the communication cost of the split is counted. Its pairing needs only lightweight profiling and neighbour information, it scales to 100 agents with up to 71% shorter training, and it offers convergence guarantees for split models. Its limitations are that heterogeneity is simulated on one server, the experiments cover image classification rather than multi-agent control, the greedy pairing is heuristic with one partner per slow agent, and malicious agents are not considered.

3.5 Summary of the Reviewed Papers

Table 3.5 summarises the four works; the next chapter compares them in detail.

Table 3.5: Summary of the reviewed works

Work	Problem	Technique	Headline result
Shabir et al. [1]	Offloading in vehicular fog computing	Multi-agent A3C with two-tier vehicle–fog federation	58.6% lower residence time and 57.11% lower delay at $\lambda = 200$
Andong and Min [2]	Cross-layer 6G edge resource management	DRQN agents, cross-layer actions, ECDH/AES secure aggregation	96.83% reliability; 0.023 J/task; 90% reliability with 50 agents
Muller et al. [3]	Federated MADL for wireless sensing	Survey of MADRL, FL/FRL, GNNs, serverless orchestration	Agenda: scalability, security, real-time safety
Mohammadabadi et al. [4]	Stragglers in serverless training	Local-loss split training, greedy pairing, AllReduce	Up to 71% shorter training at equal accuracy

Chapter 4

Discussion & Comparison

This chapter compares the four reviewed works. All concern learning across many agents without centralising their data, but they target different problems and evaluate in different environments. The comparison highlights their complementary strengths and the trade-offs among learning speed, communication, security, fairness, and deployability.

4.1 Analytical Focus and Methodology

The works examine federated multi-agent learning from four angles: the **decision problem** of where vehicles should offload tasks [1]; **joint cross-layer control with private model exchange** in 6G [2]; the **landscape** of formulations, architectures, and applications, including security and UAV networks [3]; and the **training process itself** for heterogeneous agents without a server [4]. Together they span the stack from application-level decisions to the mechanics of distributed training. Table 4.1 compares their methodologies.

4.2 Agent Design and Coordination Structure

The two RL papers design their agents differently. Paper [1] uses an **actor-critic** trained online with A3C: parallel environments decorrelate updates, no replay is needed, and the small action space (four destinations) suits a fast-changing environment. Paper [2] uses a **value-based** DRQN whose LSTM memory addresses partial observability explicitly, with prioritised replay and a richer composite action and multi-objective reward that reflect 6G's demands. The survey [3] points to further options—GNNs, attention, mean-field agents—that neither uses, and ComDML [4] decides where to *split* a model between devices, an idea that could help the RL papers train larger policies on weak hardware.

Table 4.1: Methodological comparison of the reviewed works

Aspect	**Shabir et al. [1]**	**Andong & Min [2]**	**Muller et al. [3]**	**Mohammadabadi et al. [4]**
Agents	Moving vehicles	6G edge devices	UEs, APs, UAVs, sensors	Heterogeneous devices
Learning task	Offloading policy (RL)	Cross-layer policy (RL)	RL, supervised, GNN	Image classification
Formulation	Per-agent MDP, residence-time reward	POMMDP, multi-objective reward	Markov games, Dec-POMDP, CTDE	Min–max training time
Agent model	A3C actor-critic	DRQN with LSTM	DRL, GNN, attention	Split ResNet + auxiliary net
Coordination	Vehicles $\rightarrow$ fog federation, asynchronous	Secure aggregation every K episodes	Client–server, hierarchical, OTA, serverless	Serverless AllReduce with pairing
Privacy	Data local; plain updates	ECDH + AES masking	Surveys poisoning and defences	Split-data protection; optional DP
Evaluation	SUMO, $\lambda = 60$–200	6G simulator, 800 episodes	Qualitative	CIFAR, CINIC; up to 100 agents

Coordination determines who talks to whom and whom everyone must trust. In [1] it is **hierarchical and asynchronous**, suited to intermittent vehicular connectivity; in [2] it is **synchronous and periodic**, with a server that sees only masked updates; in [4] it is **fully serverless**, removing the single point of failure and trust. Figure 4.2.1 places the systems on two axes: decentralisation of training and protection of individual updates. No reviewed system reaches the upper-right region—decentralised, cryptographically private, *and* robust to malicious agents—which is where future FedMARL systems should aim.

4.3 Robustness to Heterogeneity

Environmental heterogeneity is handled in [1] by aggregating across parallel environments and in [2] by recurrent memory and adaptive reward weights. **Statistical heterogeneity** (non-IID data) is tested only in [4], whose speed-up persists under Dirichlet-skewed labels. **System heterogeneity** is the core of [4], which exploits it through workload offloading, and is handled in [2] by excluding low-energy agents from aggregation.

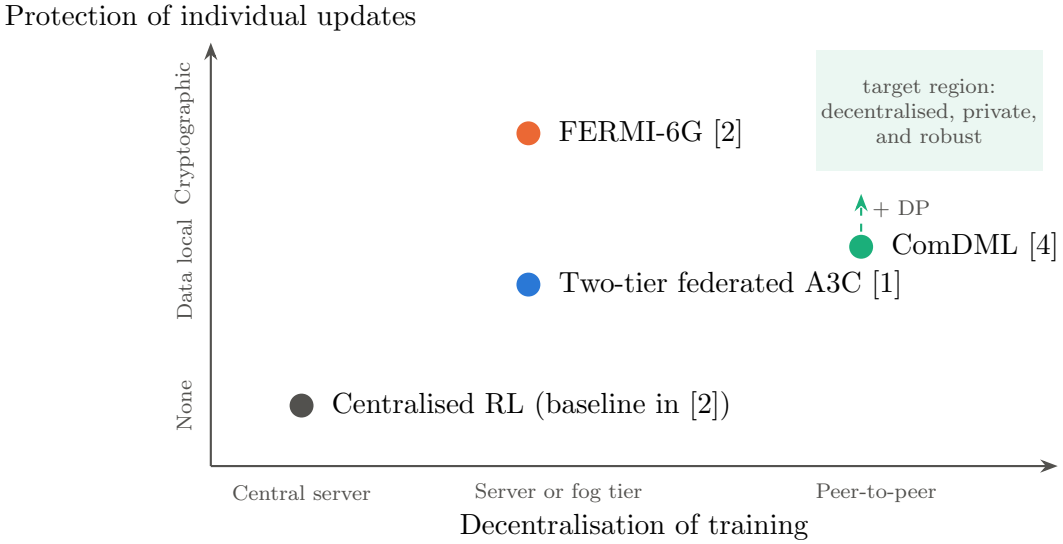


Figure 4.2.1: Qualitative design space of the reviewed systems: degree of decentralisation versus protection of individual model updates

4.4 Security and Privacy

Security differs most across the works (Table 4.2). All keep raw data local. Only [2] protects individual updates cryptographically, and only against semi-honest adversaries; [4] protects intermediate data and supports DP at a cost of a few accuracy points; [1] acknowledges that its plain updates are open to gradient spoofing; and only the survey [3] examines **malicious** participants—poisoning, backdoors, observation spoofing, and reward manipulation. The mechanisms can conflict: secure aggregation and DP hide exactly the evidence needed to detect poisoned updates. Aggregation that is both *private* and *robust*—trust-aware aggregation—is therefore essential in adversarial settings such as network security and contested UAV operations.

Table 4.2: Threats and protection mechanisms across the reviewed works

Threat	**[1]**	**[2]**	**[3]**	**[4]**
Raw-data exposure	Protected	Protected	Discussed	Protected
Inference from updates	Not protected	ECDH + AES masking	Discussed	Optional DP, data shielding
Single point of failure	Fog server	Aggregation server	Discussed	Removed
Poisoning, backdoors	Not addressed	Not addressed	Surveyed	Not addressed

4.5 Scalability, Communication, and Results

FERMI-6G keeps 90.01% reliability with 50 agents while centralised RL collapses to 12.38% [2]; ComDML keeps its advantage from 10 to 100 agents and on sparse topologies [4]; the vehicular framework has 100 vehicles, but its fog-tier scalability is argued rather than measured [1]. Communication is reduced by sharing parameters instead of state [1], by aggregating only every K episodes and letting low-energy agents skip rounds [2], and by modelling communication time explicitly in pairing [4]; ComDML's split experiment shows that ignoring communication can cancel the benefit of offloading. The headline results—58.6% lower residence time [1], 96.83% reliability at one-third of centralised energy [2], and up to 71% shorter training [4]—measure different quantities and are not directly comparable. They also come with costs: FERMI-6G has higher offloading delay, lower spectral efficiency, and lower fairness than the centralised cross-layer controller, and the vehicular scheme has higher transmission delay than greedy offloading.

4.6 Real-World Deployability

All three technical works rely on simulation—SUMO mobility with a simplified channel [1], a detailed but laptop-scale 6G simulator with five agents [2], and simulated heterogeneity on a GPU server [4]—and none reports a hardware testbed. Deployment would also require tolerating dropped participants, authenticating devices, and integrating with network management such as O-RAN [3].

4.7 Overall Comparative Insights

A consistent pattern emerges. **First**, federating locally trained models rather than raw data lets agents learn faster and more robustly than independent learning, whether they are vehicles, 6G devices, or trainers of a shared model. **Second**, this costs coordination overhead and, where a server is used, a single point of trust. **Third**, domain-specific design—residence-time rewards, cross-layer actions, profiled model splits—captures each problem's structure. **Fourth**, security has mainly meant privacy against curious observers; robustness to malicious participants is largely unaddressed. Hybrid architectures with trust-aware aggregation are therefore the most promising path forward.

Chapter 5

Applications

Federated multi-agent learning is relevant wherever many autonomous entities must coordinate without sharing raw data. This chapter outlines the main applications and relates them to the reviewed works.

5.1 Connected and Autonomous Vehicles

Vehicles generate delay-sensitive workloads such as perception, route planning, and cooperative driving, and vehicular fog computing lets them share idle resources. The base paper [1] shows that federated multi-agent DRL can decide where each task should run, sharply reducing residence time and failures under heavy load. The same approach extends to V2X channel and power selection, roadside caching, and platoon coordination. Federation matters because trajectories and requests are privacy-sensitive and no vehicle stays connected to one server for long.

5.2 6G Edge Networks and Mobile Edge Computing

In ultra-dense 6G networks, devices must choose offloading, channels, and CPU speed under URLLC, eMBB, and mMTC requirements and limited batteries. FERMI-6G [2] shows that federated agents can learn these decisions jointly and scale to 50 agents with 90% reliability while secure aggregation protects each device's model. Related applications include slicing-aware offloading, constrained multi-agent RAN slicing in O-RAN, and federated cooperative edge caching [3].

5.3 Network Security and Intrusion Detection

Intrusion detection in sensor and IoT networks needs models trained on confidential traffic spread across many nodes. Federated learning lets nodes collaboratively train detectors, including for unseen attacks, without revealing traffic, and multi-agent formula-

tions let them coordinate responses such as isolating compromised nodes [3]. The learning process, however, becomes part of the attack surface: a few compromised nodes can poison the shared detector or implant a backdoor. Secure aggregation [2] and robust aggregation [3] are therefore directly relevant, and security is where trust-aware aggregation is most urgently needed.

5.4 UAV Trajectory Planning and Aerial Networks

UAVs serve as flying base stations, relays, and data collectors, and their trajectories determine coverage, throughput, fairness, and energy use jointly with power, association, and multiple-access decisions. Federated multi-agent DRL lets a swarm learn fair communication and trajectory policies while each UAV keeps its own experience, and federated DRL has optimised trajectory and beamforming together in space–aerial–terrestrial relay networks [3]. Federation suits UAVs because their links are intermittent and bandwidth-limited, and ComDML-style balancing [4] could let small UAVs offload training to ground stations.

5.5 Sensing, Smart Cities, and Collaborative Training

In ISAC and perceptive networks, agents allocate beams between sensing and communication and fuse detections for tracking, and federated training enables collaborative recognition without centralising raw sensing streams [3]. Similar patterns arise in smart cities, industrial IoT, swarm robotics, and healthcare. Finally, collaborative training is itself an application: ComDML [4] shows that heterogeneous participants can train ResNet-scale models up to 71% faster without a server, a capability that RL agents will need as their models grow.

Chapter 6

Challenges and Future Directions

Despite the progress shown by the reviewed works, several obstacles remain before federated multi-agent learning becomes standard practice. This chapter summarises the challenges of each work and of the field, and outlines future directions.

6.1 Challenges Identified in the Reviewed Works

Shabir et al. [1] compare only with heuristics, assume a homogeneous quasi-static channel, use two training threads, and exchange updates in plain form; the authors plan more sophisticated DRL, better sampling efficiency, and less gradient traffic. **Andong and Min** [2] trade fairness for performance, have high offloading delay and low spectral efficiency, use five agents and simplified baselines, and defend only against semi-honest adversaries; they plan wider channels, power control, massive MIMO, edge-native low-latency techniques, fairness-aware coordination, and testbed validation. **Mohammadabadi et al.** [4] simulate heterogeneity, study only image classification, use heuristic one-to-one pairing, and ignore malicious agents.

6.2 Cross-Cutting Challenges

Non-stationarity and partial observability: each agent's environment shifts as others learn, and averaging policies from different local conditions can blur behaviour that should differ. **Cost of learning**: model exchange uses the very spectrum and energy the agents manage, so update frequency, size, and participation must be optimised with the task. **Security versus privacy**: hiding updates helps privacy but hinders poisoning detection. **Fairness**: maximising overall performance can make agents selfish, yet fair sharing of spectrum, servers, and UAV coverage is often a requirement. **Heterogeneity**: synchronous rounds waste fast agents' capacity, while asynchronous ones risk

stale updates. **Simulation-to-reality gap**: results come from differing simulators, and standard open benchmarks are lacking [3]. **Safety**: URLLC control, driving, and UAV flight need guarantees that current deep RL policies cannot provide.

6.3 Future Research Directions

Trust-aware aggregation, combining secure and robust aggregation through secure computation of robust statistics, reputation-weighted averaging, or verifiable training, is the most important gap identified here, especially for security and UAV applications. **Hybrid FedMARL architectures** can combine recurrent, cross-layer agents [2], hierarchical asynchronous aggregation for mobility [1], serverless workload-balanced training [4], and graph-based or mean-field policies for scale [3]. **Communication- and energy-aware federation** can use compression, event-triggered aggregation, over-the-air computation, and battery-aware participation, treating learning as another resource to schedule. **Fairness-aware coordination** can add Jain's index or worst-agent constraints to shared rewards. **Digital twins and real testbeds** can supply CTDE training environments and validate simulated gains, and **safe, verifiable policies** with feasibility layers and robust training are needed for safety-critical use [3].

6.4 Ethical and Societal Considerations

Federated multi-agent AI can strengthen privacy by keeping data on devices, but updates can still leak information and distributed sensing can enable surveillance. Safety-relevant decisions—offloading for vehicles, UAV flight, intrusion response—must be tested for failure and manipulation, with clear responsibility. The energy cost of training and communication should also be counted.

6.5 Summary

The main challenges are non-stationarity, the cost of learning, the privacy–robustness tension, fairness, heterogeneity, the simulation-to-reality gap, and safety; progress requires trust-aware aggregation, hybrid architectures, efficient federation, and real-world validation.

Chapter 7

Conclusion

Multi-agent AI systems—connected vehicles, 6G edge devices, sensor networks, and UAV swarms—must learn to coordinate from decentralised experience without exposing raw, privacy-sensitive data. Federated learning makes this possible by sharing models instead of data, and FedMARL lets agents that each see only part of the world learn policies informed by the experience of all. This seminar reviewed four recent works that apply this idea to distinct coordination problems.

Paper [1], the base paper, proposed two-tier federated A3C learning for vehicular fog computing, reducing residence time by 58.6% and end-to-end delay by 57.11% at heavy load. **Paper [2]** introduced FERMI-6G, whose DRQN agents jointly decide offloading, channel, and CPU frequency and synchronise through ECDH-based secure aggregation, achieving 96.83% reliability, the lowest energy per task, and 90% reliability with 50 agents where centralised RL collapsed. **Paper [3]** surveyed federated multi-agent deep learning for wireless sensing, including UAV networks and intrusion detection, and set out open problems in scalability, security, and safety. **Paper [4]** presented ComDML, cutting serverless training time by up to 71% by balancing workload between heterogeneous agents. The comparison leads to four conclusions:

- **Federation accelerates multi-agent learning** across very different domains.
- **Coordination has a cost** in overhead and trust, which hierarchical, asynchronous, and serverless designs reduce in different ways.
- **Agent design must fit the domain**, through suitable rewards, actions, memory, and model splits.
- **Security must go beyond privacy**: robustness to poisoning, backdoors, and spoofing is the key open problem for network security and UAV applications.

Comparing these systems across agent design, coordination structure, robustness to heterogeneity, and real-world deployability therefore reveals a consistent pattern: federating locally trained policies rather than raw data lets systems as different as connected vehicles, edge devices, sensor networks, and UAV swarms reach faster, more robust joint behaviour than independent learning—at the cost of coordination overhead and, where a server is used, a single point of trust. Hybrid FedMARL architectures, combining domain-specific agent design with trust-aware aggregation, are the most promising direction for scalable, privacy-preserving multi-agent AI.

For practitioners designing such a system, the reviewed work suggests a practical recipe: formulate the problem as a partially observable multi-agent decision process with a reward that directly reflects the service objective, such as residence time or deadline-normalised latency; give each agent a recurrent or actor-critic model sized for its hardware; train locally and aggregate periodically through a hierarchy that matches the network, such as vehicles to fog servers or devices to edge servers; protect updates with secure aggregation and, where feasible, differential privacy; let low-energy or slow agents skip rounds or offload part of their training to faster peers; monitor fairness and robustness alongside performance; and validate in realistic simulators and testbeds before deployment. Following this recipe brings together the strengths of all four works and moves toward the goal expressed in the title of this seminar: federated learning that makes multi-agent AI faster, more private, and more trustworthy across vehicular communication, network security, and UAV trajectory planning.

References

[1] B. Shabir, A. U. Rahman, A. W. Malik, R. Buyya, and M. A. Khan, "A federated multi-agent deep reinforcement learning for vehicular fog computing," *The Journal of Supercomputing*, Springer, 2022, doi: 10.1007/s11227-022-04911-8.

[2] F. J. E. N. Andong and Q. Min, "Federated multi-agent reinforcement learning for privacy-preserving and energy-aware resource management in 6G edge networks," arXiv preprint arXiv:2509.10163, Sep. 2025.

[3] N. Muller, S. DeRosa, S. Zhang, and C. L. Huan, "Federated multi-agent deep learning and neural networks for advanced distributed sensing in wireless networks," arXiv preprint arXiv:2603.16881, 2026.

[4] S. M. S. Mohammadabadi, L. Yang, F. Yan, and J. Zhang, "Communication-efficient training workload balancing for decentralized multi-agent learning," arXiv preprint arXiv:2405.00839, May 2024.

[5] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[6] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, 2016, pp. 1928–1937.

[7] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1175–1191.

[8] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[9] M. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPs," in *AAAI Fall Symposium Series*, 2015.

[10] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.